Playmaker: game logic

Code at commit c7da7b9 2026-11-05. Built 2026-11-05. Line numbers are the real ones in each file.

The rules engine: everything that decides what is legal and what happens. The computer players, the solver, the web page and the research tools are not here; the computer documents cover the computer players.

How it fits together. `scenarios.py` builds a starting `GameState` (`state.py`). `actions.legal_actions(state)` lists every legal action. `resolve.apply(state, action)` returns the next state, or a `ChanceNode` when a d20 roll decides it (each outcome with its probability). `scoring.score(state)` values a finished possession. Every switchable rule reads `state.ctx.rules` (`rules_config.Rules`); the defaults are the rules as ruled.

Contents

Section	Lines	What it is
rules_config.py	172	Every rule setting and constant (clock, tokens, limits, zones, the open-question switches).
board.py	250	The court grid: squares, zones, distances, movement shapes, arrow-line geometry.
geometry.py	118	Arrow lines and contest position (step 3h settings; defaults reproduce the current rules).
cards.py	82	Player cards: ratings and ▲ / ▼ bands, teams.
data/data.py	189	Card data: every player card and its ratings.
data/teams.py	17	The prescribed teams.
data/scenarios.py	70	The 16 scenario cards.
data/court_zones.json	240	The zone of every square (rim, three, mid, long two, wood).
scenarios.py	134	Starting positions: scenario cards and normal games, setup checks.
state.py	203	The game state (an immutable snapshot of a possession).
actions.py	380	Legal actions: movement, passes, shots, screens, dribbles, steals, interceptions, the reaction.
resolve.py	423	Applying an action: tokens, turns and phases, rolls, locks, passes, contests, turnovers.
scoring.py	236	Shot values, contests, fouls and free throws, the score and its bands.
timer.py	69	The optional decision timer and its token forfeits.
__init__.py	12	Package entry.

playmaker_engine/rules_config.py

Every rule setting and constant (clock, tokens, limits, zones, the open-question switches).

```
"""Every number and every open ruling in Playmaker lives here.

Numbers come straight from the rulebook, the trackers and the scenario rules.
Settings marked OQ-n match the numbered items in OPEN_QUESTIONS.md: each default
is the one the designer asked for, and a ruling is a one-line change here.
"""
from dataclasses import dataclass, field, replace

# ---- fixed numbers from the rules -------------------------------------------
BOARD_COLS = 17            # A..Q
BOARD_ROWS = 14            # 1..14, row 1 is the baseline
RIM = (8, 2)               # I2
TOKENS_PER_PHASE = 6
PHASE_SECONDS = 2
START_CLOCK = 12
DIE = 20                   # d20
FAR_CONTEST_PENALTY = 3    # defender further from the rim than the shooter
TURNOVER_POINTS = -5
STEAL_FOUL_BONUS = 5       # first foul on a steal
FT_MULT_TWO = 2
FT_MULT_THREE = 3
MAX_ACTIONS_PER_TURN = 2
MOVE_STRAIGHT = 3          # up to 3 straight
MOVE_DIAGONAL = 2          # up to 2 diagonal
INBOUND_PHASES = 2
SCENARIO_GOAL = 30         # an Excellent look

# Per-phase limits. 'move' is once per player; the rest are per side.
LIMITS = {
    'O': {'qs': 2, 'pass': 2, 'screen': 2, 'dribble': 2, 'hesitate': 1, 'shot': 1},
    'X': {'qs': 2, 'intercept': 2, 'challenge': 2, 'steal': 2, 'pause': 1},
}
TURN_ENDERS = {'pass', 'screen', 'dribble', 'hesitate', 'shot',
               'intercept', 'challenge', 'steal', 'pause'}

# Prestige bands, highest first: (name, lowest score in band)
BANDS = [('Excellent', 30), ('Great', 25), ('Good', 20), ('Okay', 15), ('Poor', None)]
# Coach's Call bonus by band of the look
COACH_BONUS = {'Poor': 0, 'Okay': 2, 'Good': 5, 'Great': 8, 'Excellent': 10}

# Zone -> allowed shot types and points
ZONE_SHOTS = {
    'K': ('layup', 'dunk', 'post'),
    'M': ('mid', 'floater'),
    'L': ('long',),
```

```
    '3': ('three',),
    '.': (),
}
ZONE_POINTS = {'K': 2, 'M': 2, 'L': 2, '3': 3}


@dataclass(frozen=True)
class Rules:
    # OQ-1: ball after an intercept or steal that plays on.
    #   'play_on' : ball back to the passer / stays with the carrier, -5, play continues.
    #   'end'     : the possession ends at that point.
    turnover_play_on: str = 'play_on'
    # OQ-2: when a pass resolves.
    #   'defence_turn' : on the defence's next turn (it may Intercept); if it doesn't,
    #                    the pass completes at the start of that turn.
    #   'immediate'    : completes at once unless a defender is already on the line.
    pass_resolution: str = 'defence_turn'
    # OQ-3: Sprint length = Move length + Quick Step, in one straight line.
    sprint_orth_base: int = MOVE_STRAIGHT
    sprint_diag_base: int = MOVE_DIAGONAL
    # OQ-4: roll bounds. A target >= 20 always succeeds, <= 0 always fails.
    natural_rolls: bool = False
    # OQ-5 (ruled): a declared shot worth less than 0 scores 0, matching the forced shot.
    floor_declared_shot: bool = True
    # OQ-6: defence reaction to a declared shot may use 0, 1 or 2 actions (movement, arrow
    #   turns, Steal) from the tokens still in the pool.
    reaction_kinds: tuple = ('move', 'qs', 'sprint', 'steal')
    reaction_may_decline: bool = True
    # OQ-7: defence out of tokens ends the phase at once (as written).
    defence_empty_ends_phase: bool = True
    # OQ-8: a wait (Hesitate / Pause) may not be played on the turn straight after the
    #   other side's wait.
    wait_chain_block: bool = True
    # OQ-9: screen geometry.
    #   'from_t_or_corner' : a defender's diagonal step is banned if it starts on the T
    #                        square or cuts the T square's corner (T is one of the two
    #                        squares the diagonal passes between).
    #   'from_t_only'      : only diagonal steps that start on the T square are banned.
    screen_rule: str = 'from_t_or_corner'
    # OQ-10: arrow line. A ball carrier move is illegal only if its path goes from one
    #   side of the ray to the other (touching or stepping onto the line is fine).
    arrow_line_strict: bool = True
    # OQ-11: a 1 straight + 1 diagonal Move may go in either order.
    move_either_order: bool = True
    # OQ-12 (ruled: the defence chooses any adjacent defender): contest choice.
    #   'min_value' : defence picks the adjacent defender (or none) that minimises the
    #                 expected shot value, counting foul free throws.
    #   'nearest'   : the nearest adjacent defender (rulebook wording), ties by min value.
    #   'player'    : (RULED, step 3b) the defence chooses: contest with any adjacent defender, or not.
    contest_choice: str = 'player'
    # ---- new questions found while building (OPEN_QUESTIONS.md 13+) ----
    # OQ-13: a square is 'on the pass line' if its centre lies within this distance of
    #   the segment (0.5 = the line goes through the middle half of the square;
    #   0 = exact centre, as check_scenarios.py does).
    on_line_tolerance: float = 0.5
    # OQ-14: Quick Step may be shorter than the card value (up to 2, not exactly 2).
    qs_up_to: bool = True
    # OQ-15: a player with Quick Step 0 cannot Sprint.
    sprint_needs_qs: bool = True
    # OQ-52 (RULED): each player has one Quick Step a phase, and a Sprint uses it ("uses that player's
    #   Move and Quick Step", trackers). So no Sprint after that player's Quick Step, no Quick Step after
    #   their Sprint, no second Quick Step. The side's limit of 2 Quick Steps (Sprints count) still applies.
    qs_once_per_player: bool = True
    # Step 3h (rule test, not adopted): a defender's arrow.
    #   'single' : one arrow line (the rules as they stand).
    #   'wedge'  : two arrow lines 45 degrees apart (directions a and a+1 for arrow index a): passes,
    #              drives, steals and contest facing test either line; a Sprint sets one edge to the run
    #              direction and the defender picks the side of the other.
    arrow_mode: str = 'single'
    # Step 3h (rule test, not adopted): contest quality.
    #   'current'      : any adjacent defender contests; arrow off the shooter = a foul roll; -3 when the
    #                    contester is further from the rim than the shooter.
    #   'wedge_mirror' : facing gates the contest (shooter on an arrow line); Front if the rim square
    #                    lies in the wedge mirrored backwards, Behind if it lies in the forward wedge,
    #                    else Side (needs arrow_mode 'wedge').
    #   'rim_distance' : facing gates the contest; Front if closer to the rim than the shooter, Side if
    #                    level, Behind if further.
    #   Front: full contest. Side: a foul roll (Foul contest rating). Behind: a foul roll and the -3.
    contest_geometry: str = 'current'
    # OQ-16 (ruled): who may Challenge a screen.
    #   'on_t'     : only the defender standing on the screen's T square (the one it blocks).
    #   'adjacent' : any defender next to the screener.   'any' : any defender.
    challenge_who: str = 'on_t'
    # OQ-17 (ruled): a locked ball carrier may not Dribble: a lost dribble means pass or shoot.
    locked_can_dribble: bool = False
    # OQ-18: a Dribble win frees the carrier from that defender's arrow line for the rest
    #   of the phase.
    dribble_win_lasts: str = 'phase'
    # OQ-19: before the inbound pass lands the inbounder may only Pass.
    inbounder_pass_only: bool = True
    # OQ-20 (ruled): the far-contest -3 never takes a contest below 0.
    floor_contest_zero: bool = True
    # OQ-21: a side with tokens but no legal action forfeits its remaining tokens.
    forfeit_when_stuck: bool = True
    # OQ-22: Need a Three: a two-point shot (and its free throws) scores 0.
    three_only_ft_on_twos: bool = False
    # OQ-23: Coach's Call bonus uses the band of the look (shot value, or free throws if fouled)
```

```
    #   and also applies to a forced shot by the target.
    coach_bonus_on_forced: bool = True
    # OQ-24: the arrow-line restriction applies to defenders adjacent at the start of the
    #   movement action.
    arrow_line_adjacent_at: str = 'start'
    # OQ-25: two diagonal steps of a Move go the same way (a straight diagonal line).
    diag_move_straight: bool = True
    # OQ-27 (ruled): on a failed inbound, a tie for "furthest from the rim" is the offence's
    #   choice ('offence'); 'lowest' gives the ball to the lowest piece number.
    inbound_tie: str = 'offence'
    # OQ-30 (RULED, step 3b): the forced shot's type at clock 0. 'player' = the offence picks among the
    #   types its zone allows; 'best' = the engine picks the best expected value.
    forced_shot_choice: str = 'player'
    # OQ-35 (ruled): an inbound pass works like any other pass. The defence gets its normal
    #   turn (1 or 2 actions: e.g. Move then Intercept, Sprint then Intercept, or move two
    #   players); the pass resolves after that turn, then the phase resets to the card's clock.
    #   'after_defence_turn' (ruled) | 'defence_turn_start' (step 1: lands at the start of the turn)
    inbound_resolves: str = 'after_defence_turn'
    # OQ-34 (new rule, the and-one): once a shot is declared (or forced at clock 0), any foul
    #   scores the higher of the free throws and the shot's uncontested value. Fouling a good
    #   look can then never help the defence. False = free throws only.
    and_one: bool = True


DEFAULT_RULES = Rules()


def with_rules(**changes):
    """A copy of the default rules with some settings changed."""
    return replace(DEFAULT_RULES, **changes)
```

playmaker_engine/board.py

The court grid: squares, zones, distances, movement shapes, arrow-line geometry.

```
"""Board geometry: squares, zones, adjacency, arrows, lines and movement shapes.

Coordinates are (x, y): x = 0..16 for columns A..Q, y = 1..14 for rows (row 1 is
the baseline). Off-board inbounder squares: row 0 ('K0'), x = -1 ('@9'), x = 17 ('R9').
"""
import json
import os
from functools import lru_cache
from fractions import Fraction

from . import rules_config as RC

COLS = 'ABCDEFGHIJKLMNOPQ'
W, H = RC.BOARD_COLS, RC.BOARD_ROWS
RIM = RC.RIM

# 8 directions, index 0..7 = 0, 45, ..., 315 degrees clockwise from north (row up).
DIRS = ((0, 1), (1, 1), (1, 0), (1, -1), (0, -1), (-1, -1), (-1, 0), (-1, 1))
DIR_NAMES = ('N', 'NE', 'E', 'SE', 'S', 'SW', 'W', 'NW')
ORTH = (0, 2, 4, 6)
DIAG = (1, 3, 5, 7)


def angle_to_dir(a):
    return (a % 360) // 45


def dir_to_angle(d):
    return d * 45


def on_board(p):
    return 0 <= p[0] < W and 1 <= p[1] <= H


def sq(p):
    """Square name, including off-board inbounder names."""
    x, y = p
    if x == -1:
        c = '@'
    elif x == W:
        c = 'R'
    else:
        c = COLS[x]
    return f'{c}{y}'


def parse_sq(s):
    c, r = s[0], int(s[1:])
    if c == '@':
        x = -1
    elif c == 'R':
        x = W
    else:
        x = COLS.index(c)
    return (x, r)


_ZONES_PATH = os.path.join(os.path.dirname(__file__), 'data', 'court_zones.json')
with open(_ZONES_PATH) as _f:
    _Z = json.load(_f)
ZONE = {parse_sq(k): v for k, v in _Z.items()}
```

```
63
64
65  def zone(p):
66      return ZONE.get(p, '.')
67
68
69  def cheb(a, b):
70      return max(abs(a[0] - b[0]), abs(a[1] - b[1]))
71
72
73  def dist_rim(p):
74      return cheb(p, RIM)
75
76
77  def adjacent(a, b):
78      return cheb(a, b) == 1
79
80
81  def add(p, d, n=1):
82      return (p[0] + d[0] * n, p[1] + d[1] * n)
83
84
85  def dir_between(a, b):
86      """Exact direction index from a to an adjacent square b, else None."""
87      d = (b[0] - a[0], b[1] - a[1])
88      return DIRS.index(d) if d in DIRS else None
89
90
91  def nearest_dir(frm, to):
92      """Direction closest to pointing from frm at to (ported from check_scenarios.py)."""
93      import math
94      vx, vy = to[0] - frm[0], to[1] - frm[1]
95      a = math.degrees(math.atan2(vx, vy)) % 360
96      return min(range(8), key=lambda d: min(abs(d * 45 - a), 360 - abs(d * 45 - a)))
97
98
99  # ---- lines ----------------------------------------------------------------
100
101 def _orient(p, q, r):
102     return (q[0] - p[0]) * (r[1] - p[1]) - (q[1] - p[1]) * (r[0] - p[0])
103
104
105 def seg_cross(p, q, a, b):
106     """Strict crossing of segments pq and ab (ported from check_scenarios.py)."""
107     d1, d2, d3, d4 = _orient(a, b, p), _orient(a, b, q), _orient(p, q, a), _orient(p, q, b)
108     return d1 * d2 < 0 and d3 * d4 < 0
109
110
111 def ray_end(p, d):
112     """Where the arrow ray from p in direction d leaves the board (board edge at half squares)."""
113     dx, dy = DIRS[d]
114     ts = []
115     if dx > 0:
116         ts.append(Fraction(2 * (W - 1) + 1, 2) - p[0])
117     elif dx < 0:
118         ts.append(p[0] + Fraction(1, 2))
119     if dy > 0:
120         ts.append(Fraction(2 * H + 1, 2) - p[1])
121     elif dy < 0:
122         ts.append(p[1] - Fraction(1, 2))
123     t = min(ts)
124     return (p[0] + dx * t, p[1] + dy * t)
125
126
127 def arrow_crosses(def_pos, d, a, b):
128     """Does the arrow line (ray to the board edge) strictly cross segment ab?"""
129     return seg_cross(def_pos, ray_end(def_pos, d), a, b)
130
131
132 def on_segment(p, a, b, tol):
133     """Is square p on the pass segment a->b (centre within tol of it, strictly between ends)?"""
134     if p == a or p == b:
135         return False
136     vx, vy = b[0] - a[0], b[1] - a[1]
137     wx, wy = p[0] - a[0], p[1] - a[1]
138     L2 = vx * vx + vy * vy
139     dot = wx * vx + wy * vy
140     if dot <= 0 or dot >= L2:
141         return False
142     cross = vx * wy - vy * wx
143     if tol == 0:
144         return cross == 0
145     t = Fraction(tol).limit_denominator(1000)
146     return Fraction(cross * cross) < t * t * L2
147
148
149 def path_crosses_ray(points, D, d, strict=True):
150     """Does the polyline `points` cross the arrow ray from D in direction d?
151
152     Crossing means going from one side of the ray to the other. Touching the ray,
153     or stepping onto it and back to the same side, is allowed. With strict=False,
154     stepping onto the ray at all counts as crossing.
155     """
156     dx, dy = DIRS[d]
157
158     def side(p):
159         return dx * (p[1] - D[1]) - dy * (p[0] - D[0])
```

```
    def fwd(p):
        return dx * (p[0] - D[0]) + dy * (p[1] - D[1])

    last_i, last_s = None, 0
    for i, p in enumerate(points):
        s = side(p)
        if s == 0:
            if not strict and fwd(p) > 0 and i > 0:
                return True
            continue
        if last_s and (s > 0) != (last_s > 0):
            if i - last_i == 1:
                a, b = points[last_i], p
                sa, sb = abs(last_s), abs(s)
                ix = a[0] + Fraction((b[0] - a[0]) * sa, sa + sb)
                iy = a[1] + Fraction((b[1] - a[1]) * sa, sa + sb)
                if dx * (ix - D[0]) + dy * (iy - D[1]) >= 0:
                    return True
            else:
                if all(fwd(points[j]) > 0 for j in range(last_i + 1, i)):
                    return True
        last_i, last_s = i, s
    return False


# ---- movement shapes ------------------------------------------------------

def straight_paths(p, dirs, max_len, min_len=1):
    """Paths along one straight line: list of (dir, [squares...])."""
    out = []
    for d in dirs:
        path = []
        for n in range(1, max_len + 1):
            path = path + [add(p, DIRS[d], n)]
            if n >= min_len:
                out.append((d, list(path)))
    return out


@lru_cache(maxsize=None)
def move_paths(p, rules):
    """All Move shapes from p: up to 3 straight, up to 2 diagonal, 1 straight + 1 diagonal."""
    out = [path for _, path in straight_paths(p, ORTH, RC.MOVE_STRAIGHT)]
    if rules.diag_move_straight:
        out += [path for _, path in straight_paths(p, DIAG, RC.MOVE_DIAGONAL)]
    else:
        for a in DIAG:
            m = add(p, DIRS[a])
            out.append([m])
            for b in DIAG:
                out.append([m, add(m, DIRS[b])])
    for a in ORTH:
        for b in DIAG:
            m = add(p, DIRS[a])
            out.append([m, add(m, DIRS[b])])
            if rules.move_either_order:
                m2 = add(p, DIRS[b])
                out.append([m2, add(m2, DIRS[a])])
    # drop shapes that end where they started (e.g. N then SW then... never in these shapes)
    return tuple(tuple(path) for path in out if path[-1] != p)


@lru_cache(maxsize=None)
def qs_paths(p, qs, rules):
    if qs <= 0:
        return ()
    lo = 1 if rules.qs_up_to else qs
    return tuple(tuple(path) for _, path in straight_paths(p, range(8), qs, lo))


@lru_cache(maxsize=None)
def sprint_paths(p, qs, rules):
    """(dir, path) pairs for a Sprint."""
    if rules.sprint_needs_qs and qs <= 0:
        return ()
    out = straight_paths(p, ORTH, rules.sprint_orth_base + qs)
    out += straight_paths(p, DIAG, rules.sprint_diag_base + qs)
    return tuple((d, tuple(path)) for d, path in out)


def diag_step_banned(a, b, t_sq, rules):
    """Screen geometry: is the diagonal step a->b banned by a screen whose T marks t_sq?"""
    if a[0] == b[0] or a[1] == b[1]:
        return False
    if a == t_sq:
        return True
    if rules.screen_rule == 'from_t_or_corner':
        if (a[0], b[1]) == t_sq or (b[0], a[1]) == t_sq:
            return True
    return False
```

playmaker_engine/geometry.py

Arrow lines and contest position (step 3h settings; defaults reproduce the current rules).

```python
"""Arrow lines and contest position (step 3h): one helper for every rule that reads a defender's arrow.

Two settings (rules_config): arrow_mode 'single' | 'wedge', contest_geometry 'current' | 'wedge_mirror'
| 'rim_distance'. With the defaults ('single', 'current') every function here answers exactly as the
engine did before, and the engine's callers keep their original code paths for that case.

A wedge with arrow index a has two lines, directions a and a+1 (clockwise, 45 degrees apart), so there
are 8 orientations, as with a single arrow.
"""
import math

from . import board as B
from . import rules_config as RC

FRONT, SIDE, BEHIND = 'front', 'side', 'behind'


def lines(rules, a):
    """The arrow-line directions for arrow index a."""
    if rules.arrow_mode == 'wedge':
        return (a, (a + 1) % 8)
    return (a,)


def faces(rules, a, frm, to):
    """Does the defender at frm (arrow a) have the adjacent square `to` on one of his lines?"""
    if not B.adjacent(frm, to):
        return False
    return B.dir_between(frm, to) in lines(rules, a)


def crosses(rules, pos, a, p, q):
    """Does either arrow line strictly cross segment pq (a pass line)?"""
    return any(B.arrow_crosses(pos, d, p, q) for d in lines(rules, a))


def path_crosses(rules, pts, pos, a, strict):
    """Does a movement path cross either arrow line (a drive past an adjacent defender)?"""
    return any(B.path_crosses_ray(pts, pos, d, strict=strict) for d in lines(rules, a))


def sprint_arrows(rules, run_dir):
    """Arrow indices a Sprint in direction run_dir can end with: one edge is the run direction; with a
    wedge the defender chooses the side of the other (a = run_dir, lines run_dir and run_dir+1, or
    a = run_dir-1, lines run_dir-1 and run_dir)."""
    if rules.arrow_mode == 'wedge':
        return (run_dir, (run_dir - 1) % 8)
    return (run_dir,)


def _compass(v):
    """Compass angle in degrees, clockwise from north (y up), of vector v."""
    return math.degrees(math.atan2(v[0], v[1])) % 360.0


def _in_sector(theta, start_dir, eps=1e-9):
    """Is compass angle theta within the 45-degree sector from direction start_dir clockwise to
    start_dir+1, edges included?"""
    lo = start_dir * 45.0
    d = (theta - lo) % 360.0
    return d <= 45.0 + eps or d >= 360.0 - eps


def category(rules, dpos, a, spos):
    """Front, Side or Behind for a contest from dpos (arrow a) on a shooter at spos, by the active
    geometry. Under 'current' this is descriptive only (by rim distance, as the -3 rule measures it)."""
    g = rules.contest_geometry
    if g == 'wedge_mirror':
        v = (RC.RIM[0] - dpos[0], RC.RIM[1] - dpos[1])
        if v == (0, 0):
            return FRONT                  # standing on the rim square: between shooter and basket
        th = _compass(v)
        if _in_sector(th, (a + 4) % 8):    # the wedge mirrored backwards holds the rim
            return FRONT
        if _in_sector(th, a):              # the forward wedge holds the rim
            return BEHIND
        return SIDE
    dd, ds = B.dist_rim(dpos), B.dist_rim(spos)
    return FRONT if dd < ds else SIDE if dd == ds else BEHIND


def terms(rules, dpos, a, spos):
    """How a defender at dpos with arrow a contests a shooter at spos (adjacent):
    (eligible, foul_roll, penalty, category). Under 'current' every adjacent defender is eligible, a
    foul roll means the arrow is off the shooter, and the -3 is for being further from the rim. Under
    the new geometries facing gates the contest, Side and Behind roll for a foul, Behind takes the -3."""
    on = faces(rules, a, dpos, spos)
    cat = category(rules, dpos, a, spos)
    if rules.contest_geometry == 'current':
        return True, not on, B.dist_rim(dpos) > B.dist_rim(spos), cat
    if not on:
        return False, False, False, cat
    return True, cat != FRONT, cat == BEHIND, cat


def best_terms(rules, dpos, arrows, spos):
    """The defence's best way to contest from dpos when the defender may end with any of `arrows`
```

```
    (None = any arrow, a Move was used). Returns (eligible, foul_roll, penalty, category, arrow) or None.
    Best = eligible first, then no foul roll, then no penalty."""
    cands = range(8) if arrows is None else arrows
    best = None
    for a in cands:
        t = terms(rules, dpos, a, spos)
        if not t[0]:
            continue
        key = (t[1], t[2])
        if best is None or key < best[0]:
            best = (key, t + (a,))
    return best[1] if best else None


def new_geometry(rules):
    return rules.contest_geometry != 'current'


def check(rules):
    if rules.contest_geometry == 'wedge_mirror' and rules.arrow_mode != 'wedge':
        raise ValueError("contest_geometry 'wedge_mirror' needs arrow_mode 'wedge'")
```

playmaker_engine/cards.py

Player cards: ratings and ▲ / ▼ bands, teams.

```
"""Load the player cards, teams and scenario cards from the provided data files."""
import os
import sys
import types

DATA_DIR = os.path.join(os.path.dirname(__file__), 'data')


def _load(name, extra=None):
    path = os.path.join(DATA_DIR, name + '.py')
    mod = types.ModuleType('playmaker_data_' + name)
    mod.__file__ = path
    saved = {}
    for k, v in (extra or {}).items():       # satisfy plain 'from teams import TEAMS'
        saved[k] = sys.modules.get(k)
        sys.modules[k] = v
    try:
        with open(path) as f:
            exec(compile(f.read(), path, 'exec'), mod.__dict__)
    finally:
        for k, v in saved.items():
            if v is None:
                sys.modules.pop(k, None)
            else:
                sys.modules[k] = v
    return mod


DATA = _load('data')
TEAMS_MOD = _load('teams')
SCEN_MOD = _load('scenarios', {'teams': TEAMS_MOD})

OFF, DEF = DATA.OFF, DATA.DEF


class Card:
    __slots__ = ('key', 'num', 'role', 'tier', 'tagline', 'qs', 'r', 'bands')

    def __init__(self, key, num, role, tier, tagline, qs, off, dfn):
        self.key, self.num, self.role, self.tier, self.tagline, self.qs = key, num, role, tier, tagline, qs
        vals = DATA.parse(off) + DATA.parse(dfn)
        names = OFF + DEF
        self.r = {n: v for n, (v, _) in zip(names, vals)}
        self.bands = {n: b for n, (_, b) in zip(names, vals)}

    def shot(self, t):
        return self.r['shot_' + t]

    def con(self, t):
        return self.r['con_' + t]

    def __repr__(self):
        return f'Card({self.key})'


CARDS = {row[0]: Card(*row) for row in DATA.CARDS}
TEAMS = {name: [CARDS[k] for k in keys] for name, _, keys in TEAMS_MOD.TEAMS}
TEAM_TAGLINES = {name: tag for name, tag, _ in TEAMS_MOD.TEAMS}
FIRST_GAME = TEAMS_MOD.FIRST_GAME
SCENARIO_CARDS = SCEN_MOD.SC


# ---- custom lineups (step 3d) ----------------------------------------------------------
PRESCRIBED = [name for name, _, _ in TEAMS_MOD.TEAMS]
CUSTOM = {}


def register_team(name, keys):
    """Add a custom lineup to TEAMS and return the name it is registered under. The same name with the
    same cards is reused; a name already taken by other cards gets ' (2)', ' (3)' ..."""
    keys = [str(k) for k in keys]
```

```
    if len(keys) != 5 or any(k not in CARDS for k in keys):
        raise ValueError('a custom team is five card keys')
    base = ' '.join(str(name or '').split())[:30] or 'Custom'
    n, i = base, 2
    while n in TEAMS and [c.key for c in TEAMS[n]] != keys:
        n, i = f'{base} ({i})', i + 1
    if n not in TEAMS:
        TEAMS[n] = [CARDS[k] for k in keys]
        TEAM_TAGLINES[n] = 'Custom team'
        CUSTOM[n] = keys
    return n
```

playmaker_engine/data/data.py

Card data: every player card and its ratings.

```
# Player card data, transcribed from the printed cards (the cards are the source of truth).
# Values: number + band (H above average, A average, L below average).
# Offence: dribble, pass, screen | layup, dunk, post, floater, mid, long2, three, free throw
# Defence: agility, intercept, challenge, steal, foul(steal) | layup, dunk, post, floater, mid, long2, three, foul(contest)
# qs: Quick Step from the speed colour (green 2, yellow 1, red 0)
CARDS = [
 # key, piece no, role, tier, tagline, qs, offence, defence
 ('apgA',1,'Attacking Point Guard','A','Explosive athlete. Elite finisher. Average shooter.',2,
  '19H 15A 14A | 19H 18H 11L 15H 13A 11A 10A 14A', '8H 8H 8H 4H 7A | 7A 7A 4A 5A 5A 4A 4A 9A'),
 ('apgB',1,'Attacking Point Guard','B','Quick athlete. Strong finisher. Unreliable shooter.',2,
  '17H 14A 12L | 19H 17A 10L 14H 12A 10A 9A 13L', '7A 7A 7A 3A 8A | 6L 5L 3L 4A 4A 3L 3L 10L'),
 ('apgC',1,'Attacking Point Guard','C','Decent attacker, limited shooter. Foul prone.',1,
  '16A 13L 11L | 18A 16A 9L 13H 11A 9A 8A 12L', '6A 7A 6A 3A 9L | 5L 4L 2L 3L 3L 2L 2L 11L'),
 ('cgA',1,'Combo Guard','A','Well-rounded. Above-average in most skills.',2,
  '17H 18H 15A | 18A 17A 11L 12A 13A 12H 11H 16A', '8H 8H 8H 3A 6H | 8A 8A 4A 5A 6A 5A 5A 10L'),
 ('cgB',1,'Combo Guard','B','Versatile skills. Above-average playmaking.',1,
  '15A 17H 13L | 18A 16A 10L 11A 12A 11A 10A 15A', '7A 7A 7A 3A 7A | 7A 6L 3L 4A 5A 4A 4A 11L'),
 ('cgC',1,'Combo Guard','C','Serviceable playmaker. Liability on defence.',1,
  '15A 16A 11L | 16L 15L 9L 10A 11A 10A 9A 15A', '5L 6A 5L 2A 8A | 5L 4L 2L 3L 3L 3L 3L 12L'),
 ('dpgA',1,'Defensive Point Guard','A','Elite defender. Great athlete. Average offence.',2,
  '16A 15A 16A | 18A 16A 10L 9L 11A 9A 9A 15A', '10H 9H 10H 5H 4H | 9A 10A 5A 5A 7H 6A 6H 7H'),
 ('dpgB',1,'Defensive Point Guard','B','Strong, athletic defender. Below-average offence.',1,
  '16A 14A 14A | 16L 15L 9L 8L 9L 8L 8A 14A', '9H 8H 9H 5H 5H | 8A 8A 4A 4A 6A 5A 5A 8A'),
 ('dpgC',1,'Defensive Point Guard','C','Undersized but smart defender. Poor offensive skill.',2,
  '15A 13L 12L | 16L 15L 8L 8L 9L 7L 7L 13L', '8H 7A 8H 4H 6H | 6L 6L 3L 3L 5A 4A 4A 9A'),
 ('spgA',1,'Scoring Point Guard','A','Elite scorer and ball-handler. Average defence.',2,
  '19H 16A 14A | 19H 17A 12A 15H 15H 13H 12H 18H', '7A 7A 7A 3A 7A | 7A 7A 4A 4A 4A 3L 3L 9A'),
 ('spgB',1,'Scoring Point Guard','B','Confident scorer and ball-handler. Poor defender.',1,
  '18H 15A 12L | 18A 16A 11L 14H 14H 12H 11H 17H', '6A 6A 6A 3A 8A | 6L 5L 3L 3L 3L 3L 3L 10L'),
 ('spgC',1,'Scoring Point Guard','C','Solid scorer. Good handles. Weak defence. Foul prone.',1,
  '17H 14A 11L | 18A 15L 10L 13H 13A 11A 10A 16A', '5L 6A 5L 2A 9L | 5L 4L 2L 3L 3L 3L 3L 11L'),
 ('ppgA',1,'Pass-First Point Guard','A','Elite playmaker. Strong handles. Average scoring.',2,
  '19H 20H 15A | 18A 16A 10L 11A 13A 10A 10A 15A', '8H 7A 8H 3A 6H | 8A 8A 4A 4A 5A 4A 4A 9A'),
 ('ppgB',1,'Pass-First Point Guard','B','Good playmaker. Solid handles. Limited scoring.',1,
  '18H 19H 13L | 16L 15L 9L 10A 12A 9A 9A 14A', '7A 6A 7A 3A 7A | 7A 6L 3L 3L 4A 3L 3L 10L'),
 ('ppgC',1,'Pass-First Point Guard','C','Decent playmaker. Good handles. Poor scorer.',1,
  '17H 18H 11L | 16L 15L 9L 9L 11A 8L 8A 13L', '6A 5L 6A 3A 8A | 6L 4L 2L 3L 3L 3L 3L 11L'),
 ('twA',2,'Two-Way Shooting Guard','A','Strong defender and scorer. Well-rounded skills.',1,
  '15A 15A 16A | 18A 17A 12A 11A 14H 12H 11H 16A', '9H 9H 9H 5H 5H | 9A 10A 5A 5A 7H 6A 6H 7H'),
 ('twB',2,'Two-Way Shooting Guard','B','Solid defender and scorer. Balanced skills.',1,
  '14A 15A 14A | 18A 16A 11L 10A 13A 11A 10A 15A', '8H 8H 8H 4H 6H | 8A 8A 4A 4A 6A 5A 5A 8A'),
 ('twC',2,'Two-Way Shooting Guard','C','Jack-of-all-trades. No standout skills.',1,
  '14A 13L 12L | 16L 15L 10L 9L 12A 10A 9A 15A', '7A 7A 7A 4H 7A | 6L 6L 3L 3L 5A 4A 4A 9A'),
 ('tndA',2,'Three-and-D Shooting Guard','A','Elite defender. Excellent shooter. Limited playmaking.',1,
  '14A 13L 15A | 18A 16A 11L 9L 15H 13H 12H 17H', '9H 8H 9H 5H 5H | 9A 10A 5A 5A 8H 7H 7H 7H'),
 ('tndB',2,'Three-and-D Shooting Guard','B','Good defender and shooter. Limited playmaking.',1,
  '13L 12L 13L | 16L 15L 10L 8L 14H 12H 11H 17H', '8H 8H 8H 4H 6H | 8A 8A 4A 4A 7H 6A 6H 8A'),
 ('tndC',2,'Three-and-D Shooting Guard','C','Decent at shooting and defending shooters.',1,
  '12L 11L 11L | 16L 15L 9L 8L 13A 11A 10A 16A', '7A 7A 7A 3A 7A | 6L 6L 3L 3L 6A 5A 5A 9A'),
 ('pmsA',2,'Playmaker Shooting Guard','A','Elite playmaker. Decent scorer. Average defender.',1,
  '18H 18H 15A | 18A 16A 12A 13H 13A 11A 10A 16A', '7A 7A 7A 3A 6H | 8A 8A 4A 5A 5A 4A 4A 10L'),
 ('pmsB',2,'Playmaker Shooting Guard','B','Strong playmaker. Average scorer.',1,
  '17H 17H 13L | 18A 15L 11L 11A 12A 10A 9A 15A', '6A 6A 6A 3A 7A | 7A 6L 3L 4A 4A 4A 3L 11L'),
 ('pmsC',2,'Playmaker Shooting Guard','C','Solid playmaker. Poor defender. Foul prone.',1,
  '16A 16A 11L | 16L 15L 10L 11A 11A 9A 8A 14A', '5L 6A 5L 2A 8A | 5L 4L 2L 3L 3L 3L 3L 12L'),
 ('shsA',2,'Sharpshooter Shooting Guard','A','Elite spot-up shooter. Average defender.',1,
  '15A 14A 14A | 18A 16A 11L 12A 16H 14H 13H 18H', '7A 6A 7A 2A 7A | 8A 8A 4A 4A 4A 4A 3L 9A'),
 ('shsB',2,'Sharpshooter Shooting Guard','B','Spot-up shooter. Poor playmaking. Weak defence.',1,
  '14A 13L 12L | 16L 15L 10L 11A 15H 13H 12H 17H', '6A 6A 6A 2A 8A | 7A 6L 3L 3L 3L 3L 3L 10L'),
 ('shsC',2,'Sharpshooter Shooting Guard','C','Decent spot-up shooter. Liability on defence.',1,
  '13L 12L 11L | 16L 15L 9L 10A 14H 12H 11H 16A', '5L 5L 5L 1L 9L | 5L 4L 2L 3L 3L 3L 3L 11L'),
 ('bgsA',2,'Bucket Getter Shooting Guard','A','Elite scorer. Average defender. Foul prone.',2,
  '18H 13L 14A | 19H 18H 13A 15H 15H 13H 12H 18H', '7A 7A 6A 2A 8A | 7A 7A 4A 4A 4A 4A 3L 11L'),
 ('bgsB',2,'Bucket Getter Shooting Guard','B','Strong scorer. Weak defender. Foul prone.',1,
  '17H 12L 12L | 19H 17A 12A 14H 14H 12H 11H 17H', '6A 7A 5L 2A 9L | 6L 5L 3L 3L 3L 3L 3L 12L'),
 ('bgsC',2,'Bucket Getter Shooting Guard','C','Good attacker. Poor defender. Foul prone.',1,
  '17H 11L 11L | 18A 16A 11L 13H 13A 11A 10A 16A', '5L 6A 4L 2A 10L | 5L 4L 2L 3L 3L 3L 3L 12L'),
 ('dpfA',4,'Defensive Power Forward','A','Elite defender. Athletic finisher. Poor shooter.',1,
  '14A 12L 17H | 18A 18H 15H 8L 10L 8L 7L 15A', '8H 9H 9H 4H 6H | 11H 13H 7H 5A 8H 8H 7H 5H'),
 ('dpfB',4,'Defensive Power Forward','B','Good defender. Solid finisher. Poor shooter.',0,
  '12L 11L 16A | 18A 17A 14A 8L 9L 7L 6L 14A', '7A 8H 8H 4H 7A | 10H 11H 6H 4A 7H 7H 6H 6H'),
 ('dpfC',4,'Defensive Power Forward','C','Serviceable defender. Liability on offence.',0,
  '11L 11L 15A | 16L 16A 13A 8L 9L 7L 6L 12L', '6A 7A 7A 3A 8A | 8A 10A 5A 3L 6A 6A 5A 7H'),
 ('arpfA',4,'All-Rounder Power Forward','A','Versatile scorer. Solid defender.',1,
  '15A 15A 18H | 19H 18H 16H 9L 13A 11A 9A 15A', '7A 7A 8H 3A 7A | 10H 12H 6H 5A 6A 6A 5A 7H'),
 ('arpfB',4,'All-Rounder Power Forward','B','Balanced scorer. Average defender.',1,
  '13L 14A 17H | 18A 17A 15H 8L 12A 10A 8A 14A', '6A 6A 7A 3A 8A | 9A 10A 5A 4A 5A 5A 4A 8A'),
 ('arpfC',4,'All-Rounder Power Forward','C','Decent screener. No other standout skills.',0,
  '12L 13L 16A | 16L 16A 14A 8L 11A 9A 7L 12L', '5L 5L 6A 2A 9L | 7A 9A 4A 3L 4A 4A 3L 9A'),
 ('pnpA',4,'Pick-and-Pop Power Forward','A','Elite screener. Good shooter. Foul-prone defender.',1,
  '16A 14A 19H | 18A 18H 15H 9L 14H 12H 10A 16A', '6A 7A 7A 2A 8A | 9A 11H 5A 5A 5A 5A 4A 9A'),
```

```
 ('pnpB',4,'Pick-and-Pop Power Forward','B','Strong screener. Average shooter. Poor defender.',0,
  '15A 13L 18H | 18A 17A 14A 8L 13A 11A 9A 15A', '5L 6A 6A 2A 9L | 8A 9A 4A 4A 4A 4A 3L 10L'),
 ('pnpC',4,'Pick-and-Pop Power Forward','C','Good screener. Average shooter. Poor defender.',0,
  '14A 12L 17H | 16L 16A 13A 8L 12A 10A 8A 14A', '4L 5L 5L 2A 10L | 6L 8A 3L 3L 3L 3L 3L 11L'),
 # stpfA defence: card printed colours but no numbers; values are tier B + 1 (provisional, awaiting confirmation)
 ('stpfA',4,'Stretch Power Forward','A','Strong shooter. Average, foul-prone defender.',1,
  '14A 14A 16A | 18A 17A 14A 10A 15H 13H 11H 18H', '6A 7A 7A 3A 8A | 9A 11H 5A 5A 5A 5A 4A 9A'),
 ('stpfB',4,'Stretch Power Forward','B','Good shooter. Below-average defender.',0,
  '12L 13L 15A | 16L 16A 13A 9L 14H 12H 10A 16A', '5L 6A 6A 2A 9L | 8A 9A 4A 4A 4A 4A 3L 10L'),
 ('stpfC',4,'Stretch Power Forward','C','Average shooter. Poor defender.',0,
  '11L 12L 14A | 16L 15L 12A 8L 13A 11A 9A 14A', '4L 5L 5L 2A 10L | 6L 8A 3L 3L 3L 3L 11L'),
 ('pspfA',4,'Post Scorer Power Forward','A','Dominant inside scorer. Slow, imposing defender.',0,
  '14A 13L 18H | 19H 19H 17H 9L 12A 9A 7L 15A', '6A 6A 6A 2A 9L | 10H 12H 6H 5A 6A 6A 5A 8A'),
 ('pspfB',4,'Post Scorer Power Forward','B','Strong inside scorer and screener. Slow defender.',0,
  '12L 13L 17H | 19H 18H 16H 8L 11A 8L 6L 13L', '5L 6A 5L 2A 10L | 9A 10A 5A 4A 5A 5A 4A 9A'),
 ('pspfC',4,'Post Scorer Power Forward','C','Decent inside scorer. Poor shooter and defender.',0,
  '11L 13L 16A | 18A 17A 15H 8L 10L 7L 6L 12L', '4L 5L 4L 1L 11L | 7A 9A 4A 3L 4A 4A 3L 10L'),
 ('rpcA',5,'Rim Protector Centre','A','Elite defender. Strong post moves. Poor shooter.',0,
  '12L 12L 18H | 18A 18H 16H 8L 10L 7L 6L 12L', '7A 9H 8H 3A 7A | 12H 14H 8H 7H 8H 8H 6H 4H'),
 ('rpcB',5,'Rim Protector Centre','B','Strong defender. Good post moves. Poor shooter.',0,
  '11L 12L 17H | 18A 17A 15H 8L 9L 7L 6L 12L', '6A 8H 7A 2A 8A | 11H 12H 7H 6A 7H 7H 5A 5H'),
 ('rpcC',5,'Rim Protector Centre','C','Decent defender. Limited offence. Poor shooter.',0,
  '11L 11L 16A | 16L 16A 14A 8L 9L 7L 5L 10L', '5L 7A 6A 2A 9L | 9A 11H 6H 5A 6A 6A 4A 6H'),
 ('stcA',5,'Stretch Centre','A','Good shooter. Versatile skill set. Average defender.',1,
  '15A 14A 16A | 18A 17A 15H 9L 15H 12H 10A 15A', '6A 6A 7A 3A 8A | 10H 12H 6H 6A 6A 6A 5A 7H'),
 ('stcB',5,'Stretch Centre','B','Decent shooter. Average defender with poor footwork.',0,
  '14A 13L 15A | 16L 16A 14A 8L 14H 11A 9A 14A', '5L 6A 6A 3A 9L | 9A 10A 5A 5A 5A 5A 4A 8A'),
 ('stcC',5,'Stretch Centre','C','Average shooter. Poor defender. Limited athleticism.',0,
  '13L 12L 14A | 16L 15L 13A 8L 13A 10A 8A 14A', '4L 5L 5L 2A 10L | 7A 9A 4A 4A 3L 3L 3L 9A'),
 ('pmcA',5,'Playmaking Centre','A','Great passer and screener with a versatile skill set.',0,
  '16A 17H 17H | 18A 18H 16H 11A 13A 10A 8A 17H', '6A 7A 7A 4H 7A | 10H 12H 6H 5A 5A 5A 4A 8A'),
 ('pmcB',5,'Playmaking Centre','B','Good passer and screener. Struggles defending guards.',0,
  '14A 16A 16A | 18A 17A 15H 10A 12A 9A 7L 15A', '5L 6A 6A 2A 8A | 9A 10A 5A 5A 4A 4A 3L 9A'),
 ('pmcC',5,'Playmaking Centre','C','Good passer. Poor defender.',0,
  '14A 15A 15A | 16L 16A 14A 9L 11A 8L 6L 13L', '4L 5L 5L 2A 9L | 7A 9A 4A 4A 3L 3L 3L 10L'),
 ('pscA',5,'Post Scorer Centre','A','Elite inside scorer. Shot blocker. Poor perimeter skills.',0,
  '14A 13L 19H | 19H 19H 18H 9L 11A 8L 6L 14A', '5L 6A 5L 2A 9L | 11H 13H 7H 5A 6A 6A 4A 6H'),
 ('pscB',5,'Post Scorer Centre','B','Strong inside scorer and screener. Poor perimeter skills.',0,
  '13L 12L 18H | 19H 18H 17H 8L 10L 7L 6L 13L', '4L 6A 4L 2A 10L | 10H 11H 6H 4A 5A 5A 3L 7H'),
 ('pscC',5,'Post Scorer Centre','C','Inside scorer. Poor defender. Lacks perimeter skills.',0,
  '12L 11L 17H | 18A 17A 16H 8L 9L 7L 6L 11L', '3L 5L 4L 1L 11L | 8A 10A 5A 4A 4A 4A 3L 8A'),
 ('rrcA',5,'Rim Runner Centre','A','Elite, athletic finisher. Strong defender. Poor shooter.',1,
  '12L 12L 18H | 19H 19H 16H 9L 10L 8L 6L 12L', '7A 8H 6A 4H 8A | 11H 13H 7H 6A 7H 7H 5A 5H'),
 ('rrcB',5,'Rim Runner Centre','B','Strong, fast finisher. Good athlete. Poor shooter.',1,
  '11L 11L 17H | 19H 18H 15H 8L 9L 7L 6L 11L', '6A 7A 5L 3A 9L | 10H 11H 6H 5A 6A 6A 4A 6H'),
 ('rrcC',5,'Rim Runner Centre','C','Decent finisher. Average defender. Limited skill set.',1,
  '11L 11L 16A | 18A 17A 14A 8L 9L 7L 5L 10L', '5L 6A 4L 2A 10L | 8A 10A 5A 4A 5A 5A 3L 7H'),
 ('twsfA',3,'Two-Way Small Forward','A','Great defender and scorer. Versatile player.',1,
  '16A 15A 17H | 18A 17A 14A 11A 13A 11A 10A 16A', '9H 9H 9H 4H 6H | 10H 11H 6H 5A 7H 7H 6H 8A'),
 ('twsfB',3,'Two-Way Small Forward','B','Good defender. Well-rounded skills. Solid athlete.',1,
  '15A 14A 15A | 18A 16A 13A 10A 12A 10A 9A 15A', '8H 8H 8H 3A 7A | 9A 9A 5A 4A 6A 6A 5A 9A'),
 ('stsfA',3,'Stretch Small Forward','A','Excellent shooter. Serviceable all-round skills.',1,
  '15A 14A 15A | 18A 16A 13A 11A 14H 13H 12H 17H', '7A 6A 7A 3A 7A | 9A 10A 5A 5A 6A 5A 5A 9A'),
 ('stsfB',3,'Stretch Small Forward','B','Good shooter. Decent size. Average defender. Foul prone.',1,
  '15A 13L 13L | 16L 15L 12A 10A 13A 12H 11H 15A', '6A 6A 6A 2A 8A | 8A 8A 4A 4A 5A 5A 4A 10L'),
 ('dfsfA',3,'Defensive Small Forward','A','Elite defender. Great athlete. Limited on offence.',1,
  '14A 13L 17H | 18A 17A 13A 9L 11A 9A 8A 14A', '9H 9H 10H 5H 5H | 10H 12H 7H 5A 8H 8H 7H 6H'),
 ('dfsfB',3,'Defensive Small Forward','B','Strong defender. Good athlete. Poor offensive game.',1,
  '13L 12L 15A | 18A 16A 12A 8L 10L 8L 7L 14A', '8H 8H 9H 4H 6H | 9A 10A 6H 4A 7H 7H 6H 7H'),
 ('scsfA',3,'Scoring Small Forward','A','Elite, athletic scorer. Solid defender.',1,
  '18H 15A 15A | 19H 18H 15H 12A 15H 13H 11H 17H', '7A 7A 7A 3A 7A | 8A 10A 5A 5A 5A 5A 4A 9A'),
 ('scsfB',3,'Scoring Small Forward','B','Strong scorer. Good athlete. Average defender.',1,
  '17H 14A 13L | 19H 17A 14A 11A 14H 12H 10A 16A', '6A 7A 6A 2A 8A | 7A 8A 4A 4A 4A 4A 3L 10L'),
 ('pfsfA',3,'Point Forward Small Forward','A','Elite playmaker. Great athleticism. Strong scorer.',1,
  '18H 18H 17H | 18A 17A 15H 11A 13A 12H 10A 16A', '7A 7A 8H 3A 6H | 9A 10A 5A 5A 6A 6A 5A 10L'),
 ('pfsfB',3,'Point Forward Small Forward','B','Strong playmaker. Good size. Versatile scorer.',1,
  '16A 17H 15A | 18A 16A 14A 10A 12A 11A 9A 15A', '6A 7A 7A 2A 7A | 8A 8A 4A 4A 5A 5A 4A 11L'),
 # Small Forward tier C: never printed. Derived by cards/derive_sf_c.py (tier B minus the median B-to-C drop
 # across the other 20 archetypes, clamped to the printed range; tested at 85% exact on held-out archetypes).
 # Quick Step from speed: A-to-C speed drops 1 in total on every Centre archetype.
 ('twsfC',3,'Two-Way Small Forward','C','Decent defender and scorer. No standout skills.',1,
  '14A 13L 14A | 16L 15L 12A 10A 11A 9A 8A 14A', '7A 7A 7A 2A 8A | 7A 8A 4A 3L 5A 5A 4A 10L'),
 ('stsfC',3,'Stretch Small Forward','C','Decent shooter. Poor defender. Foul prone.',0,
  '14A 12L 12L | 16L 15L 11L 10A 12A 11A 10A 14A', '5L 5L 5L 1L 9L | 6L 7A 3L 3L 4A 4A 3L 11L'),
 ('dfsfC',3,'Defensive Small Forward','C','Solid defender. Poor offensive game.',1,
  '12L 11L 14A | 16L 15L 11L 8L 9L 7L 6L 13L', '7A 7A 8H 3A 7A | 7A 9A 5A 3L 6A 6A 5A 8A'),
 ('scsfC',3,'Scoring Small Forward','C','Decent scorer. Poor defender. Foul prone.',1,
  '16A 13L 12L | 18A 16A 13A 11A 13A 11A 9A 15A', '5L 6A 5L 1L 9L | 5L 7A 3L 3L 3L 3L 2L 11L'),
 ('pfsfC',3,'Point Forward Small Forward','C','Decent playmaker. Average scorer. Weak defender.',1,
  '15A 16A 14A | 16L 15L 13A 10A 11A 10A 8A 14A', '5L 6A 6A 1L 8A | 6L 7A 3L 3L 4A 4A 3L 12L'),
]
OFF = ['dribble', 'pass', 'screen', 'shot_layup', 'shot_dunk', 'shot_post', 'shot_floater', 'shot_mid', 'shot_long', 'shot_three', 'free_throw']
DEF = ['agility', 'intercept', 'challenge', 'steal', 'foul_steal', 'con_layup', 'con_dunk', 'con_post', 'con_floater', 'con_mid', 'con_long', 'con_three', 'foul_contest']
def parse(s): return [(int(t[:-1]), t[-1]) for t in s.replace('|', ' ').split()]

# Ceiling: 19 on every rating except Pass (20 allowed: a pass only rolls when an intercept contests it, so it is never a sure thing).
# RANGE TABLE (locked 2 Nov 2026). Each entry: (lowest average, lowest above average); below the first is below average.
# Every card colour is generated from this table. Layup ratings: 17->16 for the weakest finishers so finishing matters at the rim; top stays 19 (95%), no one is a sure thing.
RANGES = {'dribble': (14, 17), 'pass': (14, 17), 'screen': (14, 17), 'shot_layup': (17, 19), 'shot_dunk': (16, 18), 'shot_post': (12, 15), 'shot_floater': (10, 13), 'shot_mid': (11, 14), 'shot_long': (9, 12), 'shot_three': (8, 11), 'free_throw': (14, 17), 'agility': (6, 8), 'intercept': (6, 8), 'challenge': (6, 8), 'steal': (2, 4), 'con_layup': (7, 10), 'con_dunk': (7, 11), 'con_post': (4, 6), 'con_floater': (4, 7), 'con_mid': (4, 7), 'con_long': (4, 7), 'con_three': (4, 6)}
FOUL_RANGES = {'foul_steal': (6, 8), 'foul_contest': (7, 9)}  # lower is better: (highest above average, highest average)
def band(name, v):
```

```
    if name in FOUL_RANGES:
        h, a = FOUL_RANGES[name]; return 'H' if v <= h else ('A' if v <= a else 'L')
    a, h = RANGES[name]
    return 'H' if v >= h else ('A' if v >= a else 'L')

# Colours changed from the printed card to match the range chart (printed colour first), kept for the end-of-set cutoff review.
CHART_FIXES = [
 ('apgB','shot_dunk',17,'H','A'), ('apgB','con_layup',6,'A','L'), ('cgB','shot_long',11,'H','A'), ('ppgC','shot_long',8,'A','L'),
 ('spgA','con_mid',4,'L','A'), ('cgA','con_long',5,'H','A'), ('twB','con_mid',6,'A','H'), ('tndB','con_long',6,'A','H'),
 ('tndC','con_mid',6,'A','H'), ('dpfC','con_mid',6,'A','H'), ('dpfC','con_long',6,'A','H'), ('arpfA','con_mid',6,'A','H'),
 ('arpfA','con_long',6,'A','H'), ('pspfA','con_mid',6,'A','H'), ('pspfA','con_long',6,'A','H'),
 ('rpcB','con_floater',6,'A','H'), ('rpcC','con_mid',6,'A','H'), ('rpcC','con_long',6,'A','H'), ('stcA','con_floater',6,'A','H'),
 ('stcA','con_mid',6,'A','H'), ('stcA','con_long',6,'A','H'), ('pscA','con_mid',6,'A','H'), ('pscA','con_long',6,'A','H'),
 ('rrcA','con_floater',6,'A','H'), ('rrcB','con_mid',6,'A','H'), ('rrcB','con_long',6,'A','H'),
]
# Provisional numbers (card printed colours only): stpfA defence = tier B + 1.
CHART_FIXES += [('twsfB','con_mid',6,'A','H'), ('twsfB','con_long',6,'A','H'), ('stsfA','con_mid',6,'A','H'), ('pfsfA','con_mid',
6,'A','H'), ('pfsfA','con_long',6,'A','H')]
```

playmaker_engine/data/teams.py

The prescribed teams.

```
# Prescribed teams. Rules: one card per position; tier shape BBBBB, ABBBC or AABCC (equal A and C);
# every team covers four jobs (a creator, two shooters, a rim anchor, a point-of-attack stopper),
# except where the identity trades exactly one away; strengths within a tight band; no card used twice.
TEAMS = [
 ('Pace and Space',    'Four shooters spread it; a rim runner dives into the gap.', ['cgB','tndB','stsfB','pnpB','rrcB']),
 ('Iron Wall',         'Three stoppers and a rim protector, plus a scorer.', ['dpgB','bgsB','dfsfB','stpfB','rpcB']),
 ('Twin Towers',       'Two post scorers inside, a scoring wing and a stopper outside.', ['spgC','twA','scsfA','pspfC','pscB']),
 ('Run and Gun',       'Get out early: attack the rim before the defence is set.', ['apgB','twB','scsfB','dpfA','rrcC']),
 ('Ball Movement',     'Two passers feeding a shooter and two posts. Find the open man.', ['ppgA','shsC','pfsfB','arpfA','pscC']),
 ('Shoot and Protect', 'Elite three-and-D wings out front, a rim protector behind.', ['ppgB','tndA','stsfA','stpfC','rpcC']),
 ('Bucket Brigade',    'Pure scoring. No stopper on purpose: outscore them.', ['spgA','bgsC','scsfC','pspfA','pmcB']),
 ('Five Out',          'Everyone shoots the three, even the centre.', ['ppgC','shsA','twsfB','stpfA','stcC']),
 ('High-Low',          'A passing four up top feeds an elite post scorer below.', ['cgA','pmsC','stsfC','arpfB','pscA']),
 ('Lockdown',          'An elite two-way wing and a big who anchors the defence.', ['dpgC','tndC','twsfA','dpfB','pmcA']),
]
DROPS = {'Bucket Brigade': 'stopper'}
FIRST_GAME = ('Five Out', 'High-Low')
```

playmaker_engine/data/scenarios.py

The 16 scenario cards.

```
# Scenario cards. Board: columns A-Q (x 0-16), rows 1-14 (row 1 = baseline), rim I2.
# Inbounder squares use off-board names: below row 1 = 'K0', left of A = '@9' (x -1), right of Q = 'R9' (x 17).
from teams import TEAMS
T={n:ks for n,_,ks in TEAMS}
SC=[
 dict(n=1,title='Baseline Box',kind='INBOUNDS',clock=8,off='Bucket Brigade',dfn='Iron Wall',
  O={1:'G6',2:'K6',3:'G3',4:'K3',5:'K0'},ball=5,inb=5,
  X={1:('H5',('man',1)),2:('J5',('man',2)),3:('H3',('man',3)),4:('J3',('man',4)),5:('K1',('ball',5))},
  text='Inbound from the baseline beside the rim. The defence packs the lane, each player touching their man.'),
 dict(n=2,title='Baseline Stack',kind='INBOUNDS',clock=8,off='Run and Gun',dfn='Twin Towers',
  O={1:'L5',2:'N0',3:'L4',4:'L3',5:'L2'},ball=2,inb=2,
  X={1:('K5',('man',1)),2:('N1',('ball',2)),3:('K4',('man',3)),4:('K3',('man',4)),5:('K2',('man',5))},
  text='Inbound from the baseline corner. Four players stacked on the lane line, all defenders on the inside.'),
 dict(n=3,title='Sideline Out',kind='INBOUNDS',clock=8,off='Ball Movement',dfn='Shoot and Protect',
  O={1:'O11',2:'C9',3:'R9',4:'K10',5:'J6'},ball=3,inb=3,
  X={1:('O10',('deny',1)),2:('E7',('help',3)),3:('Q9',('ball',3)),4:('K9',('deny',4)),5:('J5',('man',5))},
  text='Inbound from the right wing. The weak-side defender is two passes away and sags into the lane.'),
 dict(n=4,title='Deny Everything',kind='INBOUNDS',clock=8,off='Pace and Space',dfn='Lockdown',
  O={1:'O11',2:'R8',3:'L10',4:'M6',5:'J3'},ball=2,inb=2,
  X={1:('N10',('deny',1)),2:('Q8',('ball',2)),3:('L9',('deny',3)),4:('N6',('deny',4)),5:('K3',('deny',5))},
  text='Sideline inbound against a defence denying every lane. No pass is free: you must move to get open.'),
 dict(n=5,title='Sideline, Late',kind='INBOUNDS',clock=4,off='High-Low',dfn='Five Out',
  O={1:'D11',2:'@9',3:'A4',4:'F8',5:'G3'},ball=2,inb=2,
  X={1:('E10',('deny',1)),2:('A9',('ball',2)),3:('B5',('man',3)),4:('E7',('deny',4)),5:('H3',('man',5))},
  text='Inbound from the left wing, then 4 seconds: two phases to catch and score.'),
 dict(n=6,title='Need a Three',kind='LATE CLOCK',clock=6,off='Five Out',dfn='Iron Wall',
  O={1:'I11',2:'C9',3:'O9',4:'A3',5:'Q3'},ball=1,
  X={1:('I10',('ball',1)),2:('D9',('man',2)),3:('N9',('man',3)),4:('B3',('man',4)),5:('P3',('man',5))},
  text='Down 3. Only a three counts, and the defence knows it: nobody helps, everybody stays home.'),
 dict(n=7,title='Two or Three',kind='LATE CLOCK',clock=4,off='Shoot and Protect',dfn='Bucket Brigade',
  O={1:'J11',2:'B8',3:'P8',4:'F10',5:'H3'},ball=1,
  X={1:('J10',('ball',1)),2:('C8',('deny',2)),3:('O8',('deny',3)),4:('F9',('man',4)),5:('I3',('man',5))},
  text='Down 2. An Excellent three wins it; an Excellent two ties it.'),
 dict(n=8,title="Coach's Call",kind="COACH'S CALL",clock=8,off='Twin Towers',dfn='Pace and Space',
  O={1:'I11',2:'C9',3:'O9',4:'G4',5:'K3'},ball=1,target=True,
  X={1:('I10',('ball',1)),2:('D8',('deny',2)),3:('N8',('deny',3)),4:('G3',('man',4)),5:('K2',('man',5))},
  text='Before setup, secretly write the player you want to score. Run the play to get them the shot.'),
 dict(n=9,title="Coach's Call, Late",kind="COACH'S CALL",clock=6,off='Lockdown',dfn='Ball Movement',
  O={1:'E10',2:'B6',3:'N9',4:'G3',5:'K6'},ball=5,target=True,
  X={1:('F9',('deny',1)),2:('C6',('man',2)),3:('M8',('deny',3)),4:('H3',('man',4)),5:('K5',('ball',5))},
  text='Secret target with 6 seconds left. The ball is with your centre at the elbow.'),
 dict(n=10,title='Late Clock Reset',kind='LATE CLOCK',clock=4,off='Iron Wall',dfn='Run and Gun',
  O={1:'I11',2:'D9',3:'N9',4:'F7',5:'J3'},ball=1,screen={5:180},
  X={1:('I10',('ball',1)),2:('E8',('deny',2)),3:('M8',('deny',3)),4:('G6',('man',4)),5:('J2',('man',5))},
  text='The set broke down. Both Screens are spent: O4\'s is down, O5\'s is still up. Offence has 4 tokens left, defence 4.'),
 dict(n=11,title='After a Timeout',kind="COACH'S CALL",clock=8,off='Ball Movement',dfn='High-Low',
  O={1:'R10',2:'B8',3:'L11',4:'K7',5:'F5'},ball=1,inb=1,target=True,
  X={1:('Q10',('ball',1)),2:('C8',('man',2)),3:('L10',('deny',3)),4:('L7',('deny',4)),5:('G4',('help',1))},
  text='Sideline inbound after a timeout, with a secret target. Draw it up: the defence has to guess.'),
 dict(n=12,title='Mismatch on the Switch',kind='CONSTRAINT',clock=6,off='Twin Towers',dfn='Run and Gun',
  O={1:'I11',2:'C9',3:'O9',4:'L5',5:'H3'},ball=1,
  X={1:('H2',('man',5)),2:('D8',('man',2)),3:('N8',('man',3)),4:('L4',('man',4)),5:('I10',('ball',1))},
```

```
  text='A switch left X1, the point guard, on your post scorer, and X5 on the ball. The defence knows too.'),
 dict(n=13,title='One Phase Left',kind='LATE CLOCK',clock=2,off='Shoot and Protect',dfn='Lockdown',
  O={1:'I11',2:'C9',3:'O9',4:'M10',5:'G3'},ball=3,
  X={1:('I10',('man',1)),2:('D8',('man',2)),3:('N8',('ball',3)),4:('M9',('man',4)),5:('H3',('man',5))},
  text='Six tokens each, then the clock is gone. Every action has to count.'),
 dict(n=14,title='Hold for One',kind='CONSTRAINT',clock=8,off='Pace and Space',dfn='High-Low',
  O={1:'I12',2:'C9',3:'O9',4:'E10',5:'K4'},ball=1,
  X={1:('I11',('ball',1)),2:('D8',('man',2)),3:('N8',('man',3)),4:('F9',('man',4)),5:('J3',('man',5))},
  text='Last shot of the quarter. No shot before the final phase (clock on 2). A turnover ends it at 0.'),
 dict(n=15,title='In the Bonus',kind='CONSTRAINT',clock=8,off='High-Low',dfn='Bucket Brigade',
  O={1:'I11',2:'C9',3:'O9',4:'K6',5:'G3'},ball=1,
  X={1:('I10',('ball',1)),2:('D9',('man',2)),3:('N9',('man',3)),4:('K5',('man',4)),5:('H3',('man',5))},
  text='The defence has one steal foul: start on +5, and its next steal foul gives free throws. The goal is still a 30+ look.'),
 dict(n=16,title='Star Locked',kind='CONSTRAINT',clock=6,off='Bucket Brigade',dfn='Iron Wall',
  O={1:'I11',2:'C9',3:'O9',4:'G4',5:'L6'},ball=1,locked=1,
  X={1:('I10',('ball',1)),2:('D8',('man',2)),3:('N8',('man',3)),4:('H4',('man',4)),5:('K5',('man',5))},
  text='Your star picked up the ball and is locked until the phase resets. He can still pass or shoot.'),
]
```

playmaker_engine/data/court_zones.json

The zone of every square (rim, three, mid, long two, wood).

```
{
"A1": ".",
"A10": ".",
"A11": ".",
"A12": ".",
"A13": ".",
"A14": ".",
"A2": "3",
"A3": "3",
"A4": "3",
"A5": "3",
"A6": "3",
"A7": "3",
"A8": ".",
"A9": ".",
"B1": ".",
"B10": ".",
"B11": ".",
"B12": ".",
"B13": ".",
"B14": ".",
"B2": "L",
"B3": "L",
"B4": "L",
"B5": "L",
"B6": "L",
"B7": "3",
"B8": "3",
"B9": ".",
"C1": ".",
"C10": ".",
"C11": ".",
"C12": ".",
"C13": ".",
"C14": ".",
"C2": "L",
"C3": "L",
"C4": "L",
"C5": "L",
"C6": "L",
"C7": "L",
"C8": "3",
"C9": "3",
"D1": ".",
"D10": ".",
"D11": ".",
"D12": ".",
"D13": ".",
"D14": ".",
"D2": "M",
"D3": "M",
"D4": "M",
"D5": "M",
"D6": "L",
"D7": "L",
"D8": "L",
"D9": "3",
"E1": ".",
"E10": "3",
"E11": ".",
"E12": ".",
"E13": ".",
"E14": ".",
"E2": "M",
"E3": "M",
"E4": "M",
"E5": "M",
"E6": "M",
"E7": "L",
"E8": "L",
"E9": "3",
"F1": ".",
"F10": "3",
"F11": ".",
```

```
  "F12": ".",
  "F13": ".",
  "F14": ".",
  "F2": "M",
  "F3": "M",
  "F4": "M",
  "F5": "M",
  "F6": "M",
  "F7": "M",
  "F8": "L",
  "F9": "L",
  "G1": ".",
  "G10": "3",
  "G11": ".",
  "G12": ".",
  "G13": ".",
  "G14": ".",
  "G2": "K",
  "G3": "K",
  "G4": "K",
  "G5": "M",
  "G6": "M",
  "G7": "M",
  "G8": "L",
  "G9": "L",
  "H1": ".",
  "H10": "3",
  "H11": ".",
  "H12": ".",
  "H13": ".",
  "H14": ".",
  "H2": "K",
  "H3": "K",
  "H4": "K",
  "H5": "M",
  "H6": "M",
  "H7": "M",
  "H8": "L",
  "H9": "L",
  "I1": ".",
  "I10": "3",
  "I11": ".",
  "I12": ".",
  "I13": ".",
  "I14": ".",
  "I2": "K",
  "I3": "K",
  "I4": "K",
  "I5": "M",
  "I6": "M",
  "I7": "M",
  "I8": "L",
  "I9": "L",
  "J1": ".",
  "J10": "3",
  "J11": ".",
  "J12": ".",
  "J13": ".",
  "J14": ".",
  "J2": "K",
  "J3": "K",
  "J4": "K",
  "J5": "M",
  "J6": "M",
  "J7": "M",
  "J8": "L",
  "J9": "L",
  "K1": ".",
  "K10": "3",
  "K11": ".",
  "K12": ".",
  "K13": ".",
  "K14": ".",
  "K2": "K",
  "K3": "K",
  "K4": "K",
  "K5": "M",
  "K6": "M",
  "K7": "M",
  "K8": "L",
  "K9": "L",
  "L1": ".",
  "L10": "3",
  "L11": ".",
  "L12": ".",
  "L13": ".",
  "L14": ".",
  "L2": "M",
  "L3": "M",
  "L4": "M",
  "L5": "M",
  "L6": "M",
  "L7": "M",
  "L8": "L",
  "L9": "L",
  "M1": ".",
  "M10": "3",
```

```
"M11": ".",
"M12": ".",
"M13": ".",
"M14": ".",
"M2": "M",
"M3": "M",
"M4": "M",
"M5": "M",
"M6": "M",
"M7": "L",
"M8": "L",
"M9": "3",
"N1": ".",
"N10": ".",
"N11": ".",
"N12": ".",
"N13": ".",
"N14": ".",
"N2": "M",
"N3": "M",
"N4": "M",
"N5": "M",
"N6": "L",
"N7": "L",
"N8": "L",
"N9": "3",
"O1": ".",
"O10": ".",
"O11": ".",
"O12": ".",
"O13": ".",
"O14": ".",
"O2": "L",
"O3": "L",
"O4": "L",
"O5": "L",
"O6": "L",
"O7": "L",
"O8": "3",
"O9": "3",
"P1": ".",
"P10": ".",
"P11": ".",
"P12": ".",
"P13": ".",
"P14": ".",
"P2": "L",
"P3": "L",
"P4": "L",
"P5": "L",
"P6": "L",
"P7": "3",
"P8": "3",
"P9": ".",
"Q1": ".",
"Q10": ".",
"Q11": ".",
"Q12": ".",
"Q13": ".",
"Q14": ".",
"Q2": "3",
"Q3": "3",
"Q4": "3",
"Q5": "3",
"Q6": "3",
"Q7": "3",
"Q8": ".",
"Q9": "."
}
```

playmaker_engine/scenarios.py

Starting positions: scenario cards and normal games, setup checks.

```python
"""Scenario cards and normal-game setup."""
from . import board as B
from . import rules_config as RC
from .cards import SCENARIO_CARDS, TEAMS, FIRST_GAME
from .state import Context, GameState, CI, ZERO_COUNTS

SCENARIOS = {s['n']: s for s in SCENARIO_CARDS}

# Constraints the scenario rules describe in words (Scenario_rules.pdf, CONSTRAINTS; card text).
CONSTRAINTS = {
    10: dict(tok_o=4, tok_x=4, screens_used=2),     # Late Clock Reset: both Screens spent, 4 tokens each
    15: dict(fouls=1, bonus5=1),                     # In the Bonus: one steal foul already, +5
}


def resolve_arrow(pos, intent, O, ball_pos):
    """Arrow intent ('man', 'ball', 'deny', 'help') to a direction (ported from check_scenarios.py).
    Returns (direction, error or None)."""
    mode, tgt = intent
    man = O[tgt]
    if mode in ('man', 'ball'):
        return B.nearest_dir(pos, man), None
    if mode == 'help':
```

```
        return B.nearest_dir(pos, ball_pos), None
    (ax, ay), (bx, by) = ball_pos, man
    online = ((bx - ax) * (pos[1] - ay) - (by - ay) * (pos[0] - ax) == 0
              and min(ax, bx) <= pos[0] <= max(ax, bx) and min(ay, by) <= pos[1] <= max(ay, by))
    far = lambda d: (pos[0] + B.DIRS[d][0] * 30, pos[1] + B.DIRS[d][1] * 30)
    opts = list(range(8)) if online else [d for d in range(8) if B.seg_cross(pos, far(d), ball_pos, man)]
    want = B.nearest_dir(pos, man)
    if not opts:
        return want, 'deny arrow cannot cross the lane'
    return min(opts, key=lambda d: min(abs(d - want), 8 - abs(d - want))), None


def scenario_layout(n):
    """Squares, arrows and errors for scenario n, in the same terms as check_scenarios.py."""
    sc = SCENARIOS[n]
    errs = []
    O = {k: B.parse_sq(v) for k, v in sc['O'].items()}
    occ = {}
    for k, p in O.items():
        on = B.on_board(p)
        if k == sc.get('inb'):
            if on:
                errs.append('inbounder on court')
            beside = (p[1] == 0 and 0 <= p[0] < B.W) or (p[0] in (-1, B.W) and 1 <= p[1] <= B.H)
            if not beside:
                errs.append('inbounder not beside an edge square')
        elif not on:
            errs.append(f'O{k} off board')
        occ.setdefault(p, []).append(f'O{k}')
    ball_pos = O[sc['ball']]
    X, arrows = {}, {}
    for k, (s, intent) in sc['X'].items():
        p = B.parse_sq(s)
        X[k] = p
        occ.setdefault(p, []).append(f'X{k}')
        if not B.on_board(p):
            errs.append(f'X{k} off board')
        d, e = resolve_arrow(p, intent, O, ball_pos)
        if e:
            errs.append(f'X{k} {e}')
        if intent[0] == 'ball' and B.cheb(p, O[intent[1]]) != 1:
            errs.append(f'X{k} on the ball but not adjacent')
        arrows[k] = d
    for p, who in occ.items():
        if len(who) > 1:
            errs.append('shared square ' + '+'.join(who))
    return O, X, arrows, errs


def scenario_state(n, rules=None, target=None):
    """The starting GameState for scenario card n. With a Coach's Call card and no target,
    the first decision is the offence's secret target."""
    sc = SCENARIOS[n]
    O, X, arrows, errs = scenario_layout(n)
    if errs:
        raise ValueError(f'scenario {n}: ' + '; '.join(errs))
    title = sc['title']
    inb = sc.get('inb')
    ctx = Context(TEAMS[sc['off']], TEAMS[sc['dfn']], rules, title=f"{n}. {title}", kind=sc['kind'], scenario=n,
                  card_clock=sc['clock'], inbounder=(inb - 1) if inb else -1, coach_call=bool(sc.get('target')),
                  three_only=title == 'Need a Three', two_or_three=title == 'Two or Three',
                  hold_for_one=title == 'Hold for One', off_team=sc['off'], def_team=sc['dfn'])
    kw = dict(o=tuple(O[k] for k in range(1, 6)), x=tuple(X[k] for k in range(1, 6)),
              arrows=tuple(arrows[k] for k in range(1, 6)), ball=sc['ball'] - 1, clock=sc['clock'],
              inbound=1 if inb else 0)
    if sc.get('screen'):
        kw['screens'] = tuple(sorted((k - 1, B.angle_to_dir(a), False, 0) for k, a in sc['screen'].items()))
    if sc.get('locked'):
        kw['lock_o'] = 1 << (sc['locked'] - 1)
    con = CONSTRAINTS.get(n, {})
    cnt = list(ZERO_COUNTS)
    if 'screens_used' in con:
        cnt[CI['screen']] = con['screens_used']
    kw['cnt'] = tuple(cnt)
    for key in ('tok_o', 'tok_x', 'fouls', 'bonus5'):
        if key in con:
            kw[key] = con[key]
    if ctx.coach_call:
        if target is None:
            kw['stage'] = 'target'
        else:
            kw['target'] = target
    return GameState(ctx=ctx, **kw)


def game_state(off_team=FIRST_GAME[0], def_team=FIRST_GAME[1], o_squares=None, x_squares=None, arrows=None,
               ball=0, rules=None, clock=RC.START_CLOCK, check=True):
    """A normal-game start. Squares are names ('I11'); arrows are compass names or indices."""
    ctx = Context(TEAMS[off_team], TEAMS[def_team], rules, title=f'{off_team} v {def_team}', kind='GAME',
                  off_team=off_team, def_team=def_team)
    o = tuple(B.parse_sq(s) for s in o_squares)
    x = tuple(B.parse_sq(s) for s in x_squares)
    arr = tuple(B.DIR_NAMES.index(a) if isinstance(a, str) else a for a in arrows)
    if check:
        errs = setup_errors(o, x, ball)
        if errs:
            raise ValueError('; '.join(errs))
```

```
    return GameState(ctx=ctx, o=o, x=x, arrows=arr, ball=ball, clock=clock)


def setup_errors(o, x, ball):
    errs = []
    allp = list(o) + list(x)
    if len(set(allp)) != len(allp):
        errs.append('two pieces share a square')
    if not all(B.on_board(p) for p in allp):
        errs.append('a piece is off the board')
    bp = o[ball]
    if not (B.zone(bp) == '.' and bp[1] > 1):
        errs.append('the ball must start on plain wood beyond the three-point tiles')
    return errs
```

playmaker_engine/state.py

The game state (an immutable snapshot of a possession).

```
"""GameState: an immutable, hashable snapshot of a possession, plus printers."""
from dataclasses import dataclass, field, replace

from . import board as B
from . import rules_config as RC

# Per-phase counters, in a fixed order inside GameState.cnt
COUNT_KEYS = ('o_qs', 'pass', 'screen', 'dribble', 'hesitate', 'shot',
              'x_qs', 'intercept', 'challenge', 'steal', 'pause')
CI = {k: i for i, k in enumerate(COUNT_KEYS)}
ZERO_COUNTS = (0,) * len(COUNT_KEYS)


def count_key(side, kind):
    if kind == 'qs':
        return 'o_qs' if side == 'O' else 'x_qs'
    return kind


class Context:
    """Static facts about one possession (teams, rules, scenario). Compared by identity."""

    def __init__(self, off_cards, def_cards, rules=None, title='Game', kind='GAME', scenario=None,
                 card_clock=RC.START_CLOCK, inbounder=-1, coach_call=False, three_only=False,
                 two_or_three=False, hold_for_one=False, off_team='', def_team=''):
        self.off = tuple(off_cards)
        self.dfn = tuple(def_cards)
        self.rules = rules or RC.DEFAULT_RULES
        self.title, self.kind, self.scenario = title, kind, scenario
        self.card_clock = card_clock
        self.inbounder = inbounder
        self.coach_call = coach_call
        self.three_only, self.two_or_three, self.hold_for_one = three_only, two_or_three, hold_for_one
        self.off_team, self.def_team = off_team, def_team


@dataclass(frozen=True)
class Result:
    """How the possession ended."""
    kind: str                # 'shot', 'free_throws', 'steal', 'turnover_end', 'forced'
    shooter: int = -1
    shot_type: str = ''
    points: int = 0          # 2 or 3 for the declared/forced shot, 0 if none
    rating: int = 0
    contest: int = 0
    contester: int = -1
    shot_value: int = 0      # points x (rating - contest), as scored
    free_throws: int = 0
    and_one: bool = False    # the foul scored the uncontested shot value (higher than the free throws)


@dataclass(frozen=True)
class GameState:
    ctx: Context = field(compare=False, hash=False, repr=False)
    o: tuple                 # 5 O squares
    x: tuple                 # 5 X squares
    arrows: tuple            # 5 direction indices
    ball: int                # O index of the ball carrier
    stage: str = 'play'      # 'target', 'play', 'reaction', 'over'
    side: str = 'O'
    clock: int = RC.START_CLOCK
    tok_o: int = RC.TOKENS_PER_PHASE
    tok_x: int = RC.TOKENS_PER_PHASE
    moved_o: int = 0         # bitmask: players who used their Move
    moved_x: int = 0
    cnt: tuple = ZERO_COUNTS
    turn_n: int = 0          # actions already taken this turn
    final: bool = False      # defence's final turn after the offence ran out
    lock_o: int = 0
    lock_x: int = 0
    screens: tuple = ()      # (o index, T direction, fresh, beaten-by bitmask)
    pending: tuple = None    # (passer, receiver) awaiting the defence's answer
    approach: int = -1       # defender that moved onto the pass line this turn
    shot: str = None         # declared shot type while the defence reacts
    prev_wait: str = ''      # 'H' or 'P' if the other side's last turn ended in a wait
    beaten: tuple = ()       # (o, x) pairs: dribble wins this phase
    fouls: int = 0           # steal fouls so far
    bonus5: int = 0          # +5 bonuses earned
```

```
    tos: int = 0              # turnovers
    inbound: int = 0          # inbound phase number (1 or 2), 0 when the ball is in
    target: int = -1          # Coach's Call target (0 index)
    result: Result = None
    zeroed: bool = False      # Hold for One turnover: possession scores 0
    phase: int = 1
    turn_moved: int = 0
    qs_o: int = 0             # bitmask: players who used their Quick Step this phase (a Sprint uses it too)
    qs_x: int = 0
    forced: bool = False      # the pending contest is for a forced shot at clock 0       # defenders who moved during this defence turn
(inbound pass in the air)

    # ---- helpers ----
    def count(self, side, kind):
        return self.cnt[CI[count_key(side, kind)]]

    def tokens(self, side):
        return self.tok_o if side == 'O' else self.tok_x

    def occupied(self):
        return set(self.o) | set(self.x)

    def screen_of(self, i):
        for s in self.screens:
            if s[0] == i:
                return s
        return None

    def is_screening(self, i):
        return self.screen_of(i) is not None

    def locked(self, side, i):
        return bool((self.lock_o if side == 'O' else self.lock_x) >> i & 1)

    def moved(self, side, i):
        return bool((self.moved_o if side == 'O' else self.moved_x) >> i & 1)

    def quick_stepped(self, side, i):
        return bool((self.qs_o if side == 'O' else self.qs_x) >> i & 1)

    @property
    def carrier_pos(self):
        return self.o[self.ball]

    def with_(self, **kw):
        return replace(self, **kw)

    # ---- text forms ----
    def encode(self):
        """Compact one-line encoding (stable, human-diffable)."""
        a = B.DIR_NAMES
        parts = [
            f'{self.stage}:{self.side}', f'c{self.clock}', f't{self.tok_o}/{self.tok_x}',
            'O' + ','.join(B.sq(p) for p in self.o), 'X' + ','.join(f'{B.sq(p)}{a[d]}' for p, d in zip(self.x, self.arrows)),
            f'b{self.ball + 1}', f'm{self.moved_o:02x}{self.moved_x:02x}', 'n' + ''.join(str(c) for c in self.cnt),
            f'u{self.turn_n}{"F" if self.final else ""}', f'L{self.lock_o:02x}{self.lock_x:02x}',
        ]
        if self.screens:
            parts.append('S' + ','.join(f'{i + 1}{a[d]}{"*" if f else ""}{m:x}' for i, d, f, m in self.screens))
        if self.pending:
            parts.append(f'P{self.pending[0] + 1}>{self.pending[1] + 1}' + (f'a{self.approach + 1}' if self.approach >= 0 else ''))
        if self.shot:
            parts.append('Y' + self.shot)
        if self.prev_wait:
            parts.append('W' + self.prev_wait)
        if self.beaten:
            parts.append('D' + ','.join(f'{o + 1}{x + 1}' for o, x in self.beaten))
        parts.append(f'f{self.fouls}+{self.bonus5}-{self.tos}')
        if self.inbound:
            parts.append(f'i{self.inbound}')
        if self.target >= 0:
            parts.append(f'g{self.target + 1}')
        if self.result:
            r = self.result
            parts.append(f'R{r.kind}:{r.shot_value}:{r.free_throws}')
        if self.zeroed:
            parts.append('Z')
        if self.qs_o or self.qs_x:
            parts.append(f'q{self.qs_o:02x}{self.qs_x:02x}')
        return '|'.join(parts)

    def board(self):
        """ASCII board: O1* = ball, O5T^ = screen with T to the north, X2NE = arrow, L = locked."""
        cells = {}
        for i, p in enumerate(self.o):
            s = f'O{i + 1}'
            if i == self.ball:
                s += '*'
            sc = self.screen_of(i)
            if sc:
                s += 'T' + B.DIR_NAMES[sc[1]].lower()
            if self.locked('O', i):
                s += 'L'
            cells[p] = s
        for i, p in enumerate(self.x):
            s = f'X{i + 1}{B.DIR_NAMES[self.arrows[i]]}'
            if self.locked('X', i):
```

```
                s += 'L'
            cells[p] = s
        cw = 7
        lines = []
        hdr = ('    ' + ' ' * cw + ''.join(f'{c:<{cw}}' for c in B.COLS)).rstrip()
        lines.append(hdr)
        for y in range(B.H, -1, -1):
            row = ''
            for x in range(-1, B.W + 1):
                p = (x, y)
                if p in cells:
                    row += f'{cells[p]:<{cw}}'
                elif B.on_board(p):
                    z = B.zone(p)
                    row += f'{"·" if z == "." else z:<{cw}}'
                else:
                    row += ' ' * cw
            label = f'{y:>2}' if y >= 1 else '  '
            lines.append(f'{label} {row}'.rstrip())
        lines.append(hdr)
        info = f'{self.ctx.title} | clock {self.clock} | phase {self.phase} | tokens O {self.tok_o} X {self.tok_x} | to act: {self.side} ({self.stage})'
        if self.inbound:
            info += f' | inbound phase {self.inbound}'
        lines.append(info)
        lines.append('Zones: K rim  M mid/floater  L long two  3 three  · plain wood.  O1* ball, Tx screen T side, L locked, X arrows by compass.')
        return '\n'.join(lines)

    def __str__(self):
        return self.board()
```

playmaker_engine/actions.py

Legal actions: movement, passes, shots, screens, dribbles, steals, interceptions, the reaction.

```
"""Actions and legal move generation."""
from typing import NamedTuple

from . import board as B
from . import geometry as G
from . import rules_config as RC

ORDER = ['target', 'pick', 'forced_shot', 'contest', 'end_turn', 'forfeit', 'shot', 'pass', 'dribble', 'screen', 'hesitate',
         'intercept', 'steal', 'challenge', 'pause', 'move', 'qs', 'sprint']
TOKEN_FREE = {'target', 'pick', 'end_turn', 'forfeit', 'contest', 'forced_shot'}
MOVEMENT = {'move', 'qs', 'sprint'}


class Action(NamedTuple):
    """An action. A NamedTuple: hashable, ordered, cheap to build."""
    kind: str
    side: str
    piece: int = -1          # acting player (index 0..4)
    dest: tuple = None       # movement destination
    arrow: int = -1          # defender arrow after a Move
    target: int = -1         # pass receiver / dribble defender / screen T direction / challenged screener
    shot: str = ''           # shot type

    def sort_key(self):
        return (ORDER.index(self.kind), self.piece, self.dest or (-9, -9), self.arrow, self.target, self.shot)

    def __str__(self):
        who = f'{self.side}{self.piece + 1}' if self.piece >= 0 else self.side
        k = self.kind
        if k == 'move':
            s = f'{who} Move to {B.sq(self.dest)}'
            return s + (f' facing {B.DIR_NAMES[self.arrow]}' if self.arrow >= 0 else '')
        if k == 'qs':
            return f'{who} Quick Step to {B.sq(self.dest)}'
        if k == 'sprint':
            return f'{who} Sprint to {B.sq(self.dest)}'
        if k == 'pass':
            return f'{who} Pass to O{self.target + 1}'
        if k == 'screen':
            return f'{who} Screen, T {B.DIR_NAMES[self.target]}'
        if k == 'dribble':
            return f'{who} Dribble vs X{self.target + 1}'
        if k == 'shot':
            return f'{who} Shot ({self.shot})'
        if k == 'intercept':
            return f'{who} Intercept'
        if k == 'challenge':
            return f'{who} Challenge screen of O{self.target + 1}'
        if k == 'steal':
            return f'{who} Steal'
        if k == 'target':
            return f"Coach's Call target O{self.target + 1}"
        if k == 'contest':
            return f'{who} contests' if self.piece >= 0 else "X: don't contest"
        if k == 'forced_shot':
            return f'{who} forced shot ({self.shot})'
        if k == 'pick':
            return f'Failed inbound: ball to O{self.target + 1}'
        if k == 'end_turn':
            return f'{who} End turn'
        if k == 'forfeit':
```

```
            return f'{who} Forfeit tokens (no legal action)'
        return f'{who} {k.capitalize()}'

    __repr__ = __str__


_new = tuple.__new__      # fast Action construction for the big movement lists


# ---- geometry helpers that need the state ----------------------------------------

def t_square(state, screen):
    i, d = screen[0], screen[1]
    return B.add(state.o[i], B.DIRS[d])


def path_ok(state, side, i, path, rules, occ=None):
    """Is this movement path legal for player i of side?"""
    start = state.o[i] if side == 'O' else state.x[i]
    if occ is None:
        occ = state.occupied()
    for q in path:
        if not B.on_board(q) or q in occ:
            return False
    if side == 'X':
        if not state.screens:
            return True
        pts = (start,) + tuple(path)
        for scr in state.screens:
            tsq = t_square(state, scr)
            if scr[3] >> i & 1:              # beat this screen: may cross T, not stop on it
                if path[-1] == tsq:
                    return False
                continue
            for a, b in zip(pts, pts[1:]):
                if B.diag_step_banned(a, b, tsq, rules):
                    return False
    elif i == state.ball:
        pts = (start,) + tuple(path)
        for j, xp in enumerate(state.x):
            if B.adjacent(xp, start) and (i, j) not in state.beaten:
                if G.path_crosses(rules, pts, xp, state.arrows[j], rules.arrow_line_strict):
                    return False
    return True


def movement_dests(state, side, i, kind, rules):
    """dest -> run direction (Sprint) or None, for one movement kind."""
    card = (state.ctx.off if side == 'O' else state.ctx.dfn)[i]
    start = state.o[i] if side == 'O' else state.x[i]
    if kind == 'move':
        paths = [(None, p) for p in B.move_paths(start, rules)]
    elif kind == 'qs':
        paths = [(None, p) for p in B.qs_paths(start, card.qs, rules)]
    else:
        paths = B.sprint_paths(start, card.qs, rules)
    out = {}
    occ = state.occupied()
    for d, path in paths:
        dest = path[-1]
        if dest in out:
            continue
        if path_ok(state, side, i, path, rules, occ):
            out[dest] = d
    return out


def can_move_kind(state, side, i, kind):
    """Limits and status for a movement kind (ignores geometry)."""
    rules = state.ctx.rules
    if state.tokens(side) <= 0:
        return False
    if side == 'O':
        if state.is_screening(i):
            return False
        if i == state.ctx.inbounder and state.inbound:      # inbounder can't move until the ball is in
            return False
    if kind in ('move', 'sprint'):
        if state.moved(side, i) or state.locked(side, i):
            return False
    if kind in ('qs', 'sprint'):
        if state.count(side, 'qs') >= RC.LIMITS[side]['qs']:
            return False
        if rules.qs_once_per_player and state.quick_stepped(side, i):
            return False                    # OQ-52: that player's Quick Step is used (a Sprint uses it too)
    return True


def movement_actions(state, side, rules):
    """Movement actions grouped by kind (move, qs, sprint), each in piece order."""
    acts = []
    pieces = state.o if side == 'O' else state.x
    for kind in ('move', 'qs', 'sprint'):
        for i in range(5):
            if not can_move_kind(state, side, i, kind):
                continue
            dests = movement_dests(state, side, i, kind, rules)
```

```
            if side == 'X' and kind == 'move':
                start = pieces[i]
                cur = state.arrows[i]
                acts += [_new(Action, ('move', 'X', i, start, d, -1, '')) for d in range(8) if d != cur]   # arrow turn
                acts += [_new(Action, ('move', 'X', i, dest, d, -1, '')) for dest in dests for d in range(8)]
            else:
                acts += [_new(Action, (kind, side, i, dest, -1, -1, '')) for dest in dests]
                if side == 'X' and kind == 'sprint' and rules.arrow_mode == 'wedge':
                    # wedge: the run direction is one edge; the other may go either side (-1 = clockwise)
                    start = pieces[i]
                    for dest in dests:
                        r = sprint_dir(start, dest)
                        acts.append(_new(Action, (kind, side, i, dest, (r - 1) % 8, -1, '')))
    return acts


def sprint_dir(start, dest):
    dx, dy = dest[0] - start[0], dest[1] - start[1]
    n = max(abs(dx), abs(dy))
    return B.DIRS.index((dx // n, dy // n))


# ---- intercept geometry ------------------------------------------------------

def intercept_status(state, j):
    """'direct', 'approach' or None for defender j against the pending pass."""
    rules = state.ctx.rules
    passer, recv = state.pending
    a, b = state.o[passer], state.o[recv]
    p = state.x[j]
    on = B.on_segment(p, a, b, rules.on_line_tolerance)
    if B.adjacent(p, a):
        if G.faces(rules, state.arrows[j], p, a):
            return 'direct' if on else None
        return 'approach' if G.crosses(rules, p, state.arrows[j], a, b) else None
    if on:
        return 'direct'
    if G.crosses(rules, p, state.arrows[j], a, b):
        return 'approach'
    return None


def is_inbound_pass(state):
    return bool(state.pending) and state.inbound and state.pending[0] == state.ctx.inbounder


def inbound_held(state):
    """An inbound pass in the air that resolves only after the defence's whole turn (OQ-35)."""
    return is_inbound_pass(state) and state.ctx.rules.inbound_resolves == 'after_defence_turn'


def completed_view(state):
    """The state as if the pending pass had landed (used for legality only)."""
    return state.with_(ball=state.pending[1], pending=None, approach=-1)


# ---- legal actions -----------------------------------------------------------

def _offence_actions(s, rules):
    acts = []
    ctx = s.ctx
    tok = s.tok_o
    if tok <= 0:
        return acts
    carrier = s.ball
    cpos = s.o[carrier]
    inbounder_has_it = s.inbound and carrier == ctx.inbounder
    # Pass
    if s.count('O', 'pass') < RC.LIMITS['O']['pass']:
        for r in range(5):
            if r != carrier:
                acts.append(Action('pass', 'O', carrier, target=r))
    pass_only = inbounder_has_it and rules.inbounder_pass_only
    # Shot
    z = B.zone(cpos)
    if (not pass_only and B.on_board(cpos) and z != '.' and s.count('O', 'shot') < RC.LIMITS['O']['shot']
            and not (ctx.hold_for_one and s.clock > RC.PHASE_SECONDS)):
        for t in RC.ZONE_SHOTS[z]:
            acts.append(Action('shot', 'O', carrier, shot=t))
    # Dribble
    if (not pass_only and s.count('O', 'dribble') < RC.LIMITS['O']['dribble']
            and (rules.locked_can_dribble or not s.locked('O', carrier))):
        for j, xp in enumerate(s.x):
            if B.adjacent(xp, cpos):
                acts.append(Action('dribble', 'O', carrier, target=j))
    # Screen
    if s.count('O', 'screen') < RC.LIMITS['O']['screen']:
        for i, p in enumerate(s.o):
            if not B.on_board(p) or s.is_screening(i):
                continue
            if i == ctx.inbounder and s.inbound:
                continue
            for d in range(8):
                if B.on_board(B.add(p, B.DIRS[d])):
                    acts.append(Action('screen', 'O', i, target=d))
    # Hesitate
    if s.count('O', 'hesitate') < RC.LIMITS['O']['hesitate'] and not (rules.wait_chain_block and s.prev_wait == 'P'):
```

```
        acts.append(Action('hesitate', 'O'))
    acts += movement_actions(s, 'O', rules)
    return acts


def _defence_actions(s, rules, kinds=None):
    acts = []
    if s.tok_x <= 0:
        return acts
    cpos = s.o[s.ball]
    ok = (lambda k: True) if kinds is None else (lambda k: k in kinds)
    if ok('steal') and s.count('X', 'steal') < RC.LIMITS['X']['steal']:
        for j, xp in enumerate(s.x):
            if G.faces(rules, s.arrows[j], xp, cpos):
                acts.append(Action('steal', 'X', j))
    if ok('challenge') and s.count('X', 'challenge') < RC.LIMITS['X']['challenge']:
        for scr in s.screens:
            for j, xp in enumerate(s.x):
                if scr[3] >> j & 1:
                    continue
                if rules.challenge_who == 'adjacent' and not B.adjacent(xp, s.o[scr[0]]):
                    continue
                if rules.challenge_who == 'on_t' and xp != t_square(s, scr):
                    continue
                acts.append(Action('challenge', 'X', j, target=scr[0]))
    if ok('pause') and s.count('X', 'pause') < RC.LIMITS['X']['pause'] and not (rules.wait_chain_block and s.prev_wait == 'H'):
        acts.append(Action('pause', 'X'))
    mv = movement_actions(s, 'X', rules)
    if kinds is not None:
        mv = [a for a in mv if a.kind in kinds]
    return acts + mv


def legal_actions(state):
    """Every legal action for the side to act, in a deterministic order."""
    s = state
    rules = s.ctx.rules
    if s.stage == 'over':
        return []
    if s.stage == 'target':
        return [Action('target', 'O', target=i) for i in range(5)]
    if s.stage == 'contest':
        from .scoring import contest_options
        return [Action('contest', 'X', o.defender) for o in contest_options(s, s.ball, s.shot, s.forced)]
    if s.stage == 'forced':
        return [Action('forced_shot', 'O', s.ball, shot=t) for t in RC.ZONE_SHOTS[B.zone(s.o[s.ball])]]
    if s.stage == 'pick':
        return [Action('pick', 'O', target=i) for i in inbound_tie_candidates(s)]
    side = s.side
    acts = []
    if s.stage == 'reaction':
        acts = _defence_actions(s, rules, kinds=set(rules.reaction_kinds))
        if s.turn_n >= 1 or rules.reaction_may_decline or not acts:
            acts.append(Action('end_turn', 'X'))
    elif side == 'O':
        acts = _offence_actions(s, rules)
        if s.turn_n >= 1:
            acts.append(Action('end_turn', 'O'))
    else:
        if s.pending:
            can_icpt = s.count('X', 'intercept') < RC.LIMITS['X']['intercept'] and s.tok_x > 0
            if s.turn_n == 0 and can_icpt:
                for j in range(5):
                    if intercept_status(s, j) == 'direct':
                        acts.append(Action('intercept', 'X', j))
            if s.turn_n == 1 and s.approach >= 0 and can_icpt:
                acts.append(Action('intercept', 'X', s.approach))
            if inbound_held(s):
                # the defence plays a normal turn while the ball is in the air; a defender already
                # on the line (and still there, unmoved) may intercept with either action
                if s.turn_n == 1 and can_icpt:
                    for j in range(5):
                        if intercept_status(s, j) == 'direct' and not (s.turn_moved >> j & 1):
                            acts.append(Action('intercept', 'X', j))
                acts += [a for a in _defence_actions(s, rules) if a.kind != 'steal']
                if s.turn_n >= 1:
                    acts.append(Action('end_turn', 'X'))
            elif is_inbound_pass(s):
                if s.turn_n == 0 and can_icpt:
                    for a in movement_actions(s, 'X', rules):
                        if is_approach(s, a):
                            acts.append(a)
                acts.append(Action('end_turn', 'X'))       # let the inbound pass in
            else:
                acts += _defence_actions(completed_view(s), rules)
                if s.turn_n >= 1:
                    acts.append(Action('end_turn', 'X'))
        else:
            acts = _defence_actions(s, rules)
            if s.turn_n >= 1:
                acts.append(Action('end_turn', 'X'))
    if not acts and rules.forfeit_when_stuck and s.tokens(side) > 0:
        acts = [Action('forfeit', side)]
    return _ordered(acts)


def _ordered(acts):
```

```
    """Group by kind in ORDER (stable within a kind: generation order is deterministic).
    Movement actions already come grouped and last, so only the rest needs sorting."""
    rest = [a for a in acts if a.kind not in MOVEMENT]
    if len(rest) == len(acts):
        return sorted(rest, key=lambda a: ORDER.index(a.kind))
    mv = [a for a in acts if a.kind in MOVEMENT]          # generated grouped by kind already
    rest.sort(key=lambda a: ORDER.index(a.kind))          # stable sort keeps generation order
    return rest + mv


def inbound_tie_candidates(state):
    """On-court players (not the inbounder) tied for furthest from the rim."""
    ib = state.ctx.inbounder
    cands = [i for i in range(5) if i != ib and B.on_board(state.o[i])]
    far = max(B.dist_rim(state.o[i]) for i in cands)
    return [i for i in cands if B.dist_rim(state.o[i]) == far]


def is_approach(state, a):
    """Is this defender movement an approach onto the pending pass line?"""
    if not (state.pending and a.side == 'X' and a.kind in MOVEMENT and state.turn_n == 0):
        return False
    if state.count('X', 'intercept') >= RC.LIMITS['X']['intercept']:
        return False
    if intercept_status(state, a.piece) != 'approach':
        return False
    passer, recv = state.pending
    return B.on_segment(a.dest, state.o[passer], state.o[recv], state.ctx.rules.on_line_tolerance)
```

playmaker_engine/resolve.py

Applying an action: tokens, turns and phases, rolls, locks, passes, contests, turnovers.

```
"""apply(): the rules in motion. Randomness is never hidden: rolls come back as ChanceNodes."""
from dataclasses import dataclass
from fractions import Fraction

from . import board as B
from . import rules_config as RC
from .actions import Action, intercept_status, is_approach, inbound_held, MOVEMENT, TOKEN_FREE
from .scoring import p_success, p_foul, choose_contest, shot_points, free_throws, contest_options, foul_on_shot_points
from .state import CI, ZERO_COUNTS, Result, count_key


@dataclass(frozen=True)
class ChanceNode:
    """outcomes: tuple of (probability, label, GameState or ChanceNode). Probabilities sum to 1.
    roller: the side rolling the d20 ('O' for opposed rolls, 'X' for the defender's foul rolls)."""
    outcomes: tuple
    what: str = ''
    roller: str = ''

    def flatten(self, prefix=''):
        out = []
        for p, label, node in self.outcomes:
            lab = f'{prefix}{label}'
            if isinstance(node, ChanceNode):
                for q, l2, s in node.flatten(lab + '/'):
                    out.append((p * q, l2, s))
            else:
                out.append((p, lab, node))
        return out

    def total(self):
        return sum(p for p, _, _ in self.flatten())


def chance(what, outcomes, roller=''):
    """Build a node, dropping impossible branches; a sure thing collapses to its state."""
    outs = tuple((Fraction(p), l, s) for p, l, s in outcomes if p > 0)
    if len(outs) == 1:
        return outs[0][2]
    return ChanceNode(outs, what, roller)


def cmap(node, fn):
    """Apply fn to every state leaf of a node (fn may itself return a ChanceNode)."""
    if isinstance(node, ChanceNode):
        return ChanceNode(tuple((p, l, cmap(s, fn)) for p, l, s in node.outcomes), node.what, node.roller)
    return fn(node)


def sample(node, rng):
    """Pick one outcome. Returns (state, label)."""
    labels = []
    while isinstance(node, ChanceNode):
        r = Fraction(rng.random())
        acc = Fraction(0)
        pick = node.outcomes[-1]
        for o in node.outcomes:
            acc += o[0]
            if r < acc:
                pick = o
                break
        labels.append(pick[1])
        node = pick[2]
    return node, '/'.join(labels)
```

```python


def force(node, *labels):
    """Follow named outcomes (for tests and replays)."""
    for lab in labels:
        if not isinstance(node, ChanceNode):
            break
        for p, l, s in node.outcomes:
            if l == lab:
                node = s
                break
        else:
            raise KeyError(f'no outcome {lab!r} in {[l for _, l, _ in node.outcomes]}')
    return node


# ---- small state edits ------------------------------------------------------

def _inc(s, side, kind):
    c = list(s.cnt)
    c[CI[count_key(side, kind)]] += 1
    return s.with_(cnt=tuple(c))


def _spend(s, side):
    return s.with_(tok_o=s.tok_o - 1) if side == 'O' else s.with_(tok_x=s.tok_x - 1)


def _lock(s, side, i):
    return s.with_(lock_o=s.lock_o | 1 << i) if side == 'O' else s.with_(lock_x=s.lock_x | 1 << i)


def _terminal(s, result, zeroed=False):
    return s.with_(stage='over', side='-', result=result, zeroed=s.zeroed or zeroed, pending=None, shot=None)


def _turnover(s):
    """A turnover that plays on (OQ-1), or ends the possession."""
    s = s.with_(tos=s.tos + 1)
    if s.ctx.hold_for_one:
        return _terminal(s, Result('turnover_end'), zeroed=True)
    if s.ctx.rules.turnover_play_on == 'end':
        return _terminal(s, Result('turnover_end'))
    return s


def new_phase(s, clock):
    return s.with_(clock=clock, tok_o=RC.TOKENS_PER_PHASE, tok_x=RC.TOKENS_PER_PHASE, moved_o=0, moved_x=0,
                   cnt=ZERO_COUNTS, turn_n=0, final=False, lock_o=0, lock_x=0, approach=-1, prev_wait='',
                   beaten=(), side='O', phase=s.phase + 1, pending=None, turn_moved=0, qs_o=0, qs_x=0)


def complete_pass(s):
    passer, recv = s.pending
    s = s.with_(ball=recv, pending=None, approach=-1)
    if s.inbound and passer == s.ctx.inbounder:
        # The ball is in: reset to a new phase with the card's clock.
        s = new_phase(s.with_(inbound=0), s.ctx.card_clock)
    return s


def resolve_shot(s, forced=False):
    """The declared (or forced) shot reaches its contest. With contest_choice='player' this returns the
    decision node (the defence chooses who contests); otherwise the automatic contest is resolved."""
    shooter = s.ball
    spos = s.o[shooter]
    rules = s.ctx.rules
    if forced:
        if not B.on_board(spos) or B.zone(spos) == '.':
            return _terminal(s, Result('forced', shooter, '', 0))
        types = RC.ZONE_SHOTS[B.zone(spos)]
        if rules.forced_shot_choice == 'player' and len(types) > 1:
            return s.with_(stage='forced', side='O', shot=None, forced=True, turn_n=0, pending=None)
        if rules.forced_shot_choice == 'player':
            t = types[0]
        else:
            t = max(types, key=lambda t: (choose_contest(s, shooter, t, True).ev, -types.index(t)))
        s = s.with_(shot=t, forced=True)
    if rules.contest_choice == 'player':
        return s.with_(stage='contest', side='X', turn_n=0, forced=forced, pending=None)
    return contest_outcome(s, choose_contest(s, shooter, s.shot, forced), forced)


def contest_outcome(s, opt, forced):
    """The result of one contest option: a terminal state, or a foul roll (arrow off the shooter)."""
    shooter = s.ball
    spos = s.o[shooter]
    card = s.ctx.off[shooter]
    t = s.shot
    pts = shot_points(spos)
    kind = 'forced' if forced else 'shot'
    made = _terminal(s, Result(kind, shooter, t, pts, card.shot(t), opt.contest, opt.defender, opt.value, 0))
    if opt.clean or opt.p_foul == 0:
        return made
    _, and1 = foul_on_shot_points(card, t, pts, s.ctx.rules)
    fouled = _terminal(s, Result('free_throws', shooter, t, pts, card.shot(t), opt.contest, opt.defender, 0, opt.ft, and1))
    fr = s.ctx.dfn[opt.defender].r['foul_contest']
```

```
    why = 'arrow off the shooter' if s.ctx.rules.contest_geometry == 'current' else f'{opt.category} contest'
    return chance(f'Contest foul {fr} (X{opt.defender + 1}, {why})',
                  [(opt.p_foul, 'foul', fouled), (1 - opt.p_foul, 'no foul', made)], 'X')


def settle(node):
    """Resolve contest and forced-shot decisions the automatic way (defence's minimum, offence's best type).
    For analysis code written before those became player decisions; play itself never needs it."""
    if isinstance(node, ChanceNode):
        return ChanceNode(tuple((p, l, settle(sub)) for p, l, sub in node.outcomes), node.what, node.roller)
    if node.stage == 'contest':
        return settle(contest_outcome(node, choose_contest(node, node.ball, node.shot, node.forced), node.forced))
    if node.stage == 'forced':
        types = RC.ZONE_SHOTS[B.zone(node.o[node.ball])]
        t = max(types, key=lambda t: (choose_contest(node, node.ball, t, True).ev, -types.index(t)))
        return settle(apply(node, Action('forced_shot', 'O', node.ball, shot=t)))
    return node


def end_phase(s):
    if s.stage == 'over':
        return s
    if s.pending:
        phase = s.phase
        s = complete_pass(s)
        if s.phase != phase:              # an inbound pass landed: that already started a new phase
            return s
    if s.inbound:
        if s.inbound < RC.INBOUND_PHASES:
            return new_phase(s, s.clock).with_(inbound=s.inbound + 1)
        # Not in after two phases: turnover, -5, play on; ball to the player furthest from the rim.
        s = s.with_(tos=s.tos + 1)
        if s.ctx.hold_for_one:
            return _terminal(s, Result('turnover_end'), zeroed=True)
        ib = s.ctx.inbounder
        cands = [i for i in range(5) if i != ib and B.on_board(s.o[i])]
        far = max(B.dist_rim(s.o[i]) for i in cands)
        tied = [i for i in cands if B.dist_rim(s.o[i]) == far]
        s = new_phase(s.with_(inbound=0), s.ctx.card_clock)
        if len(tied) > 1 and s.ctx.rules.inbound_tie == 'offence':
            return s.with_(stage='pick')                # the offence picks who gets the ball
        return s.with_(ball=tied[0])
    clock = s.clock - RC.PHASE_SECONDS
    if clock <= 0:
        s = s.with_(clock=0, tos=s.tos + 1, phase=s.phase)
        if s.ctx.hold_for_one:
            return _terminal(s, Result('turnover_end'), zeroed=True)
        return resolve_shot(s, forced=True)
    return new_phase(s, clock)


def _age_screens(s):
    keep = tuple((i, d, False, m) for i, d, f, m in s.screens if f)
    return s.with_(screens=keep)


def end_turn(s, side, kind=''):
    if s.stage == 'over':
        return s
    if s.stage == 'reaction':
        return resolve_shot(s)
    if side == 'O':
        s = _age_screens(s)
        s = s.with_(side='X', turn_n=0, prev_wait='H' if kind == 'hesitate' else '', final=s.tok_o == 0, turn_moved=0)
        if s.tok_x <= 0:
            return end_phase(s)
        return s
    phase = s.phase
    if s.pending:
        s = complete_pass(s)
        if s.phase != phase:
            return s
    s = s.with_(prev_wait='P' if kind == 'pause' else '', approach=-1)
    if s.final:
        return end_phase(s)
    return s.with_(side='O', turn_n=0)


def _after(prev, s, side, kind):
    """Turn and phase bookkeeping after an action's effect."""
    if s.stage == 'over' or s.phase != prev.phase:
        return s
    if kind == 'end_turn':
        return end_turn(s, side, kind)
    if kind == 'forfeit':
        if side == 'X':
            if s.stage == 'reaction':
                return resolve_shot(s)
            return end_phase(s)
        return end_turn(s, side, kind)
    s = s.with_(turn_n=s.turn_n + 1)
    if s.stage == 'reaction':
        if kind in RC.TURN_ENDERS or s.turn_n >= RC.MAX_ACTIONS_PER_TURN or s.tok_x <= 0:
            return resolve_shot(s)
        return s
    if side == 'X' and s.tok_x <= 0 and s.ctx.rules.defence_empty_ends_phase:
        return end_phase(s)
```

```
    if kind in RC.TURN_ENDERS or s.turn_n >= RC.MAX_ACTIONS_PER_TURN or s.tokens(side) <= 0:
        return end_turn(s, side, kind)
    return s


# ---- the main entry point -------------------------------------------------------

def apply(state, action, check=False):
    """Apply an action. Returns a new GameState, or a ChanceNode listing every outcome."""
    if check:
        from .actions import legal_actions
        if action not in legal_actions(state):
            raise ValueError(f'illegal action {action} in {state.encode()}')
    s = state
    a = action
    side = a.side
    k = a.kind

    if k == 'target':
        return s.with_(target=a.target, stage='play')
    if k == 'pick':
        return s.with_(ball=a.target, stage='play')
    if k == 'forced_shot':
        s = s.with_(shot=a.shot, stage='play')
        if s.ctx.rules.contest_choice == 'player':
            return s.with_(stage='contest', side='X', forced=True)
        return contest_outcome(s, choose_contest(s, s.ball, a.shot, True), True)
    if k == 'contest':
        opts = {o.defender: o for o in contest_options(s, s.ball, s.shot, s.forced)}
        if a.piece not in opts:
            raise ValueError(f'X{a.piece + 1} cannot contest this shot')
        return contest_outcome(s, opts[a.piece], s.forced)

    if k not in TOKEN_FREE:
        s = _spend(s, side)
    if k == 'forfeit':
        s = s.with_(tok_o=0) if side == 'O' else s.with_(tok_x=0)
    if k == 'end_turn' or k == 'forfeit':
        return _after(state, s, side, k)

    # A pending pass lands at the start of the defence's turn unless this action answers it.
    if side == 'X' and s.pending and not inbound_held(state):
        answering = k == 'intercept' or (k in MOVEMENT and is_approach(state, a))
        if not answering:
            phase = s.phase
            s = complete_pass(s)
            if s.phase != phase:          # an inbound pass landed: new phase, offence to act
                return s

    if k in MOVEMENT:
        if side == 'O':
            o = list(s.o)
            o[a.piece] = a.dest
            s = s.with_(o=tuple(o))
        else:
            x = list(s.x)
            x[a.piece] = a.dest
            arr = list(s.arrows)
            if k == 'move':
                arr[a.piece] = a.arrow
            elif k == 'sprint':
                start = state.x[a.piece]
                dx, dy = a.dest[0] - start[0], a.dest[1] - start[1]
                n = max(abs(dx), abs(dy))
                run = B.DIRS.index((dx // n, dy // n))
                # wedge (step 3h): the run direction is one edge; a.arrow = run - 1 puts the other edge
                # anticlockwise, otherwise it goes clockwise (arrow index = run)
                arr[a.piece] = a.arrow if (s.ctx.rules.arrow_mode == 'wedge' and a.arrow == (run - 1) % 8) else run
            s = s.with_(x=tuple(x), arrows=tuple(arr))
            if k in MOVEMENT and is_approach(state, a):
                s = s.with_(approach=a.piece)
            if state.pending:
                s = s.with_(turn_moved=s.turn_moved | 1 << a.piece)
        if k in ('move', 'sprint'):
            s = s.with_(moved_o=s.moved_o | 1 << a.piece) if side == 'O' else s.with_(moved_x=s.moved_x | 1 << a.piece)
        if k in ('qs', 'sprint'):
            s = _inc(s, side, 'qs')
            s = s.with_(qs_o=s.qs_o | 1 << a.piece) if side == 'O' else s.with_(qs_x=s.qs_x | 1 << a.piece)
        return _after(state, s, side, k)

    s = _inc(s, side, k)
    rules = s.ctx.rules
    O, X = s.ctx.off, s.ctx.dfn

    if k == 'pass':
        s = s.with_(pending=(a.piece, a.target), approach=-1)
        if rules.pass_resolution == 'immediate':
            if not any(intercept_status(s, j) == 'direct' for j in range(5)):
                phase = s.phase
                s = complete_pass(s)
                if s.phase != phase:
                    return s
        return _after(state, s, side, k)

    if k == 'screen':
        s = s.with_(screens=tuple(sorted(s.screens + ((a.piece, a.target, True, 0),))))
        return _after(state, s, side, k)
```

```
    356
    357     if k == 'hesitate' or k == 'pause':
    358         return _after(state, s, side, k)
    359
    360     if k == 'shot':
    361         s = s.with_(stage='reaction', shot=a.shot, side='X', turn_n=0)
    362         if s.tok_x <= 0:
    363             return resolve_shot(s)
    364         return s
    365
    366     if k == 'dribble':
    367         p = p_success(O[s.ball].r['dribble'] - X[a.target].r['agility'], rules)
    368         win = _lock(s, 'X', a.target).with_(beaten=tuple(sorted(set(s.beaten) | {(s.ball, a.target)})))
    369         lose = _lock(s, 'O', s.ball)
    370         return chance(f'Dribble {O[s.ball].r["dribble"]} v Agility {X[a.target].r["agility"]} (O{s.ball + 1} v X{a.target + 1})', [
    371             (p, 'success', _after(state, win, side, k)),
    372             (1 - p, 'failure', _after(state, lose, side, k))], 'O')
    373
    374     if k == 'intercept':
    375         passer, recv = s.pending
    376         p = p_success(O[passer].r['pass'] - X[a.piece].r['intercept'], rules)
    377         ok = complete_pass(s)
    378         if ok.phase != s.phase:
    379             ok_node = ok                                  # inbound pass landed: new phase
    380         else:
    381             ok_node = _after(state, ok, side, k)
    382         lost = _turnover(s.with_(pending=None, approach=-1))   # ball stays with the passer
    383         lost_node = lost if lost.stage == 'over' else _after(state, lost, side, k)
    384         return chance(f'Pass {O[passer].r["pass"]} v Intercept {X[a.piece].r["intercept"]} (O{passer + 1} to O{recv + 1}, X{a.piece +
1})', [
    385             (p, 'success', ok_node), (1 - p, 'failure', lost_node)], 'O')
    386
    387     if k == 'challenge':
    388         scr = s.screen_of(a.target)
    389         p = p_success(O[a.target].r['screen'] - X[a.piece].r['challenge'], rules)
    390         holds = _lock(s, 'X', a.piece)
    391         fails = s.with_(screens=tuple((i, d, f, m | (1 << a.piece) if i == a.target else m) for i, d, f, m in s.screens))
    392         return chance(f'Screen {O[a.target].r["screen"]} v Challenge {X[a.piece].r["challenge"]} (O{a.target + 1}, X{a.piece + 1})', [
    393             (p, 'success', _after(state, holds, side, k)),
    394             (1 - p, 'failure', _after(state, fails, side, k))], 'O')
    395
    396     if k == 'steal':
    397         carrier = s.ball
    398         pf = p_foul(X[a.piece].r['foul_steal'], rules)
    399         pk = p_success(O[carrier].r['dribble'] - X[a.piece].r['steal'], rules)
    400         fs = s.with_(fouls=s.fouls + 1)
    401         if s.stage == 'reaction':
    402             pts = shot_points(s.o[carrier])
    403             card = O[carrier]
    404             fpts, and1 = foul_on_shot_points(card, s.shot, pts, rules)
    405             foul_node = _terminal(fs, Result('free_throws', carrier, s.shot, pts, card.shot(s.shot), 0, a.piece, 0,
    406                                              fpts, and1))
    407             stolen_node = _terminal(s.with_(tos=s.tos + 1), Result('steal', carrier, s.shot, pts),
    408                                     zeroed=s.ctx.hold_for_one)
    409         else:
    410             if fs.fouls >= 2:
    411                 foul_node = _terminal(fs, Result('free_throws', carrier, '', 2, 0, 0, a.piece, 0,
    412                                                  free_throws(O[carrier], 2)))
    413             else:
    414                 foul_node = _after(state, fs.with_(bonus5=fs.bonus5 + 1), side, k)
    415             st = _turnover(s)
    416             stolen_node = st if st.stage == 'over' else _after(state, st, side, k)
    417         kept_node = _after(state, s, side, k)
    418         inner = chance(f'Dribble {O[carrier].r["dribble"]} v Steal {X[a.piece].r["steal"]} (O{carrier + 1} v X{a.piece + 1})',
    419                        [(pk, 'kept', kept_node), (1 - pk, 'stolen', stolen_node)], 'O')
    420         return chance(f'Steal foul {X[a.piece].r["foul_steal"]} (X{a.piece + 1})',
    421                       [(pf, 'foul', foul_node), (1 - pf, 'no foul', inner)], 'X')
    422
    423     raise ValueError(f'unknown action {a}')
```

playmaker_engine/scoring.py

Shot values, contests, fouls and free throws, the score and its bands.

```
     1 """Rolls, shot values, contests, free throws, bands and the possession score."""
     2 from dataclasses import dataclass
     3 from fractions import Fraction
     4
     5 from . import board as B
     6 from . import geometry as G
     7 from . import rules_config as RC
     8
     9
    10 # ---- rolls ----------------------------------------------------------------
    11
    12 def p_success(target, rules=RC.DEFAULT_RULES):
    13     """P(d20 <= target). Defaults: >= 20 always succeeds, <= 0 always fails (OQ-4)."""
    14     if rules.natural_rolls:                    # alternative: natural 1 succeeds, 20 fails
    15         t = max(1, min(19, target))
    16         return Fraction(t, RC.DIE)
    17     t = max(0, min(RC.DIE, target))
    18     return Fraction(t, RC.DIE)
    19
    20
    21 def p_foul(foul_rating, rules=RC.DEFAULT_RULES):
    22     """Straight foul roll: d20 <= foul rating."""
    23     return p_success(foul_rating, rules)
```

```python


# ---- shots ----------------------------------------------------------------

def band(score):
    for name, lo in RC.BANDS:
        if lo is None or score >= lo:
            return name
    return 'Poor'


def shot_points(pos):
    return RC.ZONE_POINTS.get(B.zone(pos), 0)


def free_throws(card, points):
    mult = RC.FT_MULT_THREE if points == 3 else RC.FT_MULT_TWO
    return mult * card.r['free_throw']


def foul_on_shot_points(card, shot_type, pts, rules):
    """Points for a foul on a declared or forced shot: free throws, or under the and-one
    rule the higher of the free throws and the shot's uncontested value.
    Returns (points, and_one_applied)."""
    ft = free_throws(card, pts)
    if rules.and_one and shot_type:
        open_value = pts * card.shot(shot_type)
        if open_value > ft:
            return open_value, True
    return ft, False


def contest_rating(dcard, shot_type, dpos, spos, rules):
    c = dcard.con(shot_type)
    if B.dist_rim(dpos) > B.dist_rim(spos):
        c -= RC.FAR_CONTEST_PENALTY
    if rules.floor_contest_zero:
        c = max(0, c)
    return c


@dataclass(frozen=True)
class ContestOption:
    defender: int          # -1 = nobody contests
    contest: int
    clean: bool            # arrow on the shooter (no foul roll)
    p_foul: Fraction
    value: int             # shot value if no foul
    ft: int                # free throws if fouled
    ev: Fraction
    category: str = ''     # step 3h: front / side / behind (descriptive under the current geometry)


def contest_options(state, shooter, shot_type, forced=False):
    """Every contest choice open to the defence for this shot."""
    ctx, rules = state.ctx, state.ctx.rules
    spos = state.o[shooter]
    card = ctx.off[shooter]
    pts = shot_points(spos)
    rating = card.shot(shot_type)
    ft, _ = foul_on_shot_points(card, shot_type, pts, rules)
    floor = forced or rules.floor_declared_shot

    def val(c):
        v = pts * (rating - c)
        return max(0, v) if floor else v

    opts = [ContestOption(-1, 0, True, Fraction(0), val(0), ft, Fraction(val(0)))]
    for j, xp in enumerate(state.x):
        if not B.adjacent(xp, spos):
            continue
        eligible, foul_roll, penalty, cat = G.terms(rules, xp, state.arrows[j], spos)
        if not eligible:
            continue                    # step 3h geometries: arrow lines off the shooter, no contest
        if rules.contest_geometry == 'current':
            c = contest_rating(ctx.dfn[j], shot_type, xp, spos, rules)
        else:
            c = ctx.dfn[j].con(shot_type) - (RC.FAR_CONTEST_PENALTY if penalty else 0)
            if rules.floor_contest_zero:
                c = max(0, c)
        clean = not foul_roll
        pf = Fraction(0) if clean else p_foul(ctx.dfn[j].r['foul_contest'], rules)
        v = val(c)
        ev = pf * ft + (1 - pf) * v
        opts.append(ContestOption(j, c, clean, pf, v, ft, ev, cat))
    return opts


def choose_contest(state, shooter, shot_type, forced=False):
    opts = contest_options(state, shooter, shot_type, forced)
    rules = state.ctx.rules
    if rules.contest_choice == 'nearest':
        spos = state.o[shooter]
        adj = [o for o in opts if o.defender >= 0]
        if adj:
            d2 = lambda o: (state.x[o.defender][0] - spos[0]) ** 2 + (state.x[o.defender][1] - spos[1]) ** 2
            best = min(d2(o) for o in adj)
```

```
            opts = [o for o in adj if d2(o) == best]
    return min(opts, key=lambda o: (o.ev, not o.clean, o.defender))


def shot_ev(state, shooter, shot_type, forced=False):
    """Expected value of a shot taken now against the defence's best contest."""
    return choose_contest(state, shooter, shot_type, forced).ev


# ---- possession score ------------------------------------------------------

@dataclass(frozen=True)
class Score:
    shot_value: int
    free_throws: int
    bonus: int
    turnovers: int
    turnover_points: int
    coach_bonus: int
    total: int
    band: str
    look: int               # shot value, or free throws if fouled (what the goal/bonus judge)
    goal_met: bool          # scenario goal: a look of 30+
    late_clock: str         # 'win' / 'tie' / 'loss' on Two or Three, else ''
    detail: str

    def __str__(self):
        parts = [f'total {self.total} ({self.band})', f'shot value {self.shot_value}']
        if self.free_throws:
            parts.append(f'free throws {self.free_throws}')
        if self.bonus:
            parts.append(f'foul bonus +{self.bonus}')
        if self.turnovers:
            parts.append(f'turnovers {self.turnovers} ({self.turnover_points})')
        if self.coach_bonus:
            parts.append(f"coach's call +{self.coach_bonus}")
        if self.late_clock:
            parts.append(f'late clock: {self.late_clock}')
        parts.append(f'goal {"met" if self.goal_met else "missed"}')
        return ', '.join(parts) + (f'. {self.detail}' if self.detail else '')


def is_terminal(state):
    return state.stage == 'over'


def score(state):
    """Final possession score with a breakdown. Works on terminal states."""
    ctx, rules = state.ctx, state.ctx.rules
    r = state.result
    sv = ft = 0
    detail = ''
    pts = r.points if r else 0
    if r:
        sv, ft = r.shot_value, r.free_throws
        if ctx.three_only and pts != 3:
            if sv or ft:
                detail = 'Need a Three: only a three-point shot scores'
            sv = 0
            if not rules.three_only_ft_on_twos:
                ft = 0
        if r.kind in ('shot', 'forced', 'free_throws'):
            who = f'O{r.shooter + 1} {r.shot_type}'
            if r.kind == 'free_throws':
                how = 'and-one, scores the open shot' if r.and_one else 'free throws'
                detail = (detail + '. ' if detail else '') + f'{who}: fouled, {how} {r.free_throws}'
            else:
                cby = f'X{r.contester + 1}' if r.contester >= 0 else 'nobody'
                detail = (detail + '. ' if detail else '') + f'{who} {r.points} x ({r.rating} - {r.contest}) contested by {cby}'
        elif r.kind == 'steal':
            detail = 'stolen after the shot was declared'
        elif r.kind == 'turnover_end':
            detail = 'turnover ended the possession'
    look = ft if (r and r.kind == 'free_throws') else sv
    bonus = RC.STEAL_FOUL_BONUS * state.bonus5
    tpts = RC.TURNOVER_POINTS * state.tos
    coach = 0
    if ctx.coach_call and r and r.shooter >= 0 and r.shooter == state.target and r.kind in ('shot', 'forced', 'free_throws'):
        if r.kind != 'forced' or rules.coach_bonus_on_forced:
            coach = RC.COACH_BONUS[band(look)]
    total = sv + ft + bonus + tpts + coach
    if state.zeroed:
        total = 0
    late = ''
    if ctx.two_or_three:
        if r and look >= RC.SCENARIO_GOAL and pts == 3:
            late = 'win'
        elif r and look >= RC.SCENARIO_GOAL and pts == 2:
            late = 'tie'
        else:
            late = 'loss'
    goal = bool(r) and look >= RC.SCENARIO_GOAL and not state.zeroed
    return Score(sv, ft, bonus, state.tos, tpts, coach, total, band(total), look, goal, late, detail)


def late_clock_key(sc):
    """Head-to-head ranking for Late Clock cards: result first, then shot value."""
```

```
    return ({'win': 2, 'tie': 1, 'loss': 0}.get(sc.late_clock, 0), sc.look)


# ---- match ----------------------------------------------------------------

def tiebreak_probability(rating, contest, rules=RC.DEFAULT_RULES):
    """P(make) for the match tiebreaker roll: d20 <= rating - contest."""
    return p_success(rating - contest, rules)


def match_result(total_a, total_b, final_a=None, final_b=None, rules=RC.DEFAULT_RULES):
    """Winner of a two-possession match. On a tie, final_x = (rating, contest) of each
    player's final shot; returns {'A': p, 'B': p, 'tie': p} (a make beats a miss; both
    make or both miss stays tied)."""
    if total_a != total_b:
        return {'A': Fraction(int(total_a > total_b)), 'B': Fraction(int(total_b > total_a)), 'tie': Fraction(0)}
    pa = tiebreak_probability(*final_a, rules)
    pb = tiebreak_probability(*final_b, rules)
    return {'A': pa * (1 - pb), 'B': pb * (1 - pa), 'tie': pa * pb + (1 - pa) * (1 - pb)}
```

playmaker_engine/timer.py

The optional decision timer and its token forfeits.

```
"""The decision timer's one engine event (step 3d part C), off by default.

timer_forfeit(state) is an ordinary state change, logged and replayed like any other. It is never in
the legal action set and never offered to the solver.

- In play or a reaction, with tokens left: the acting side loses 1 token from its pool; the turn goes on
  (if that was its last token, the turn ends, as it does when the last token is spent on an action).
- In play or a reaction, with the pool empty: the acting side's turn ends.
- A token-free decision (the contest choice, the forced shot type, the failed-inbound pick): the default
  applies (don't contest; the first legal shot type the zone allows; the first legal pick). No token is lost.
"""
from .actions import Action, legal_actions
from .resolve import apply, end_turn, end_phase, resolve_shot

TIMED_STAGES = ('play', 'reaction', 'contest', 'forced', 'pick')
TOKEN_FREE_STAGES = ('contest', 'forced', 'pick')


def token_free(state):
    return state.stage in TOKEN_FREE_STAGES


def timer_forfeit(state):
    """Returns (node, effect): node is a GameState or a ChanceNode. effect: 'token' (1 token lost, the turn goes
    on), 'token_end' (the last token lost, so the turn ends, exactly as spending it on an action would),
    'end_turn' (the pool was already empty) or 'default' (a token-free decision's default applied)."""
    s = state
    if s.stage not in TIMED_STAGES:
        raise ValueError(f'no timer in stage {s.stage}')
    if s.stage == 'contest':
        return apply(s, Action('contest', 'X', -1)), 'default'
    if s.stage in ('forced', 'pick'):
        kind = 'forced_shot' if s.stage == 'forced' else 'pick'
        first = next(a for a in legal_actions(s) if a.kind == kind)
        return apply(s, first), 'default'
    side = s.side
    if s.tokens(side) > 0:
        s = s.with_(tok_o=s.tok_o - 1) if side == 'O' else s.with_(tok_x=s.tok_x - 1)
        if s.tokens(side) > 0:
            return s, 'token'
        return _out_of_tokens(s, side), 'token_end'        # the last token: as after spending it on an action
    return _out_of_tokens(s, side), 'end_turn'


def _out_of_tokens(s, side):
    """The same ending the engine applies when a side's last token goes (resolve._after), without counting
    the forfeit as one of the turn's actions."""
    if s.stage == 'reaction':
        return resolve_shot(s)
    if side == 'X' and s.ctx.rules.defence_empty_ends_phase:
        return end_phase(s)
    return end_turn(s, side, 'timer')


def default_action(state):
    """The action a token-free default stands for (for the log and the feed)."""
    if state.stage == 'contest':
        return Action('contest', 'X', -1)
    kind = 'forced_shot' if state.stage == 'forced' else 'pick'
    return next(a for a in legal_actions(state) if a.kind == kind)


# Timer settings (step 3d C1/C3). The web session reads these; the engine itself never runs a clock.
TIMER_CONFIG = dict(
    settings=[None, 60, 45, 30],   # seconds per decision; None = off (the default)
    grace=15,                      # admin grace after selecting an action, to pick its square / target / direction
    regrace_after=5,               # a fresh grace needs this many seconds of main timer since the last selection
    warn_at=10,                    # the ring changes pattern (and the phone may vibrate) at this many seconds
)
```

playmaker_engine/__init__.py

Package entry.

```python
"""Playmaker rules engine (step 1 of the solver programme)."""
from .rules_config import Rules, DEFAULT_RULES, with_rules
from .state import GameState, Context, Result
from .actions import Action, legal_actions
from .resolve import apply, sample, force, ChanceNode
from .scoring import score, is_terminal, Score
from .scenarios import scenario_state, game_state, SCENARIOS
from .policies import Policy, RandomPolicy, GreedyPolicy, play, format_log

__all__ = ['Rules', 'DEFAULT_RULES', 'with_rules', 'GameState', 'Context', 'Result', 'Action', 'legal_actions',
           'apply', 'sample', 'force', 'ChanceNode', 'score', 'is_terminal', 'Score', 'scenario_state',
           'game_state', 'SCENARIOS', 'Policy', 'RandomPolicy', 'GreedyPolicy', 'play', 'format_log']
```